Week 2 · Surge Speed Control

USV Guidance, Navigation and Control (Graduate) · Department of Autonomous Vehicle System Engineering, Chungnam National University | revised 2026-09-04

★ Reference material — read this first

The five courses below are **taught by the instructor of this course** and are the assumed background for it. They run from the fundamentals down to the graduate material, so start wherever the gap is. Every frame, symbol and derivation used here is developed in them at length; anyone whose prerequisites are thin should work through them first, then return.

#	Course	Level	Lang.	Video	Slides and code
1	**Control Engineering** (제어공학) — the foundation: transfer functions, feedback, stability, root locus, PID	undergraduate	KO	playlist	drive
2	**Control System Design** (제어시스템설계) — design rather than analysis: specifications, loop shaping, discrete implementation	undergraduate	KO	playlist	drive
3	**Advanced Control Engineering** (제어공학특론) — reference frames, the 6-DOF equation of motion, rotation matrices and Euler angles, linearisation and trim, vehicle control design	graduate	EN	playlist	drive
4	**Sensor Signal Processing and Fusion** (센서신호처리 및 융합) — sensor models, noise, estimation and multi-sensor fusion	graduate	EN	playlist	drive
5	**Capstone Design** (캡스톤디자인) — a vehicle project carried end to end	undergraduate	KO	playlist	drive

Not fluent in MATLAB or Simulink yet? Do these before Part 2. Every laboratory in this course is Simulink, and the Onramp courses are free and take a few hours each.

Tool	Where to start
MATLAB	MATLAB Onramp · Core MATLAB Skills
Simulink	Simulink Onramp · instructor's Simulink lectures part 1 · part 2

- **Course**: USV Guidance, Navigation and Control (Graduate)
- **Department**: Autonomous Vehicle System Engineering, Chungnam National University
- **This week**: ① the surge axis as a first-order, type 0 plant identified from the plant itself ② why proportional control leaves an error and integral action removes it ③ what an actuator limit does to an integrator, and three ways of surviving it

★ Prerequisites from the previous week

- From Week 1: $T_u = 1.1025$ s and $K_u = 0.012894$ (m/s)/N, and the propeller curve $T = k\,n|n|$.
- Two numbers, and nothing else, are carried in from **Appendix A1**: the surge force this vessel can produce lies in $X \in [-133.42,\ 239.36]$ N. That limit is what causes the windup of §2-6. Appendix A1 derives it; a reader who takes the two numbers on trust loses nothing this week.

Learning Outcomes

Upon completion of this week, the learner is able to:

1. Identify the DC gain and time constant of the surge axis from a step response, and state the discrepancy between the measurement and the first-order model.
2. Predict, from the final value theorem, the steady-state error of a proportional controller on a type 0 plant, and explain why no gain removes it.
3. Choose K_p and K_i to place the closed-loop poles at a specified ζ and ω_n, and explain why the resulting overshoot exceeds the value tabulated for that ζ.
4. State where the derivative term enters the equation of motion of a velocity loop, and predict its effect on the damping ratio before running the simulation.
5. Explain integrator windup as a consequence of actuator saturation rather than of integral action, and implement clamping and back-calculation.
6. Report a settling or recovery time with the tolerance band and the averaging window stated.
7. Build a PID controller from elementary blocks and from Simulink's PID Controller block, and state where the two differ and why.
8. Explain why a PID never differentiates directly, write the pseudo-derivative $Ns/(s+N)$ as a filter and a subtraction, and choose N from the closed-loop bandwidth rather than by making it large.

Prerequisites and Setup

Item	Requirement
MATLAB	R2024b, with Simulink
Toolboxes	Control System Toolbox, for `tf`, `feedback` and `stepinfo` in the analysis script
MSS	`Tools/MSS`, added by the setup script
Course folder	`GradCourse/lectures/W02_simulink`
Expected duration	60 min theory, 90 min laboratory

Part 1 · Theory

Where this week goes

Seven sections, answering three questions. Each answer creates the next question — that is the shape of the week, and it is worth holding on to.

	Question	Sections
1	**What does the surge axis look like to a controller?** One gain and one time constant, derived from the full model	2-1
2	**What does each term of a PID buy, and what does it cost?** P leaves an error; I removes it; D is the one that behaves differently here than it will in Week 3	2-2 to 2-4
3	**What breaks when the actuator saturates?** Windup, and why an anti-windup scheme is not an optional extra	2-5 to 2-7

The number the week turns on is $K_u = 0.012894$ (m/s)/N. Once the plant is two numbers, every steady-state claim in 2-2 and 2-3 is arithmetic rather than simulation.

The one thing to carry into Week 3: the derivative term sits beside the **mass** here, and it will sit beside the **damping** there. The term does not change; the axis does.

2-1. The plant, reduced to two numbers

- Before a controller can be designed, the thing being controlled has to be written down. This section starts from the same 6-DOF equation as Week 1 and ends with **two numbers** — a gain and a time constant — that between them describe everything the surge axis does.
- Those two numbers are what the rest of the week argues with. Once the plant is K_u and T_u, every steady-state claim in §2-2 and §2-3 is arithmetic rather than simulation.

Where the surge equation comes from

- The controller of this week acts on one scalar equation. That equation is not an assumption — it is the first row of the 3-DOF model of §1-7, and this section derives it so that the terms thrown away can be named and later reclaimed.
- Start from the horizontal-plane model, $\boldsymbol{\nu} = [u\ v\ r]^\top$ (Fossen, *Handbook*, 2nd ed., §7.3):

$$\mathbf{M}\dot{\boldsymbol{\nu}} + \mathbf{C}(\boldsymbol{\nu})\boldsymbol{\nu} + \mathbf{D}(\boldsymbol{\nu})\boldsymbol{\nu} = \boldsymbol{\tau}$$

- For a vessel symmetric about its centreline, with the body origin on that centreline and the centre of gravity offset by x_g along x_b:

$$\mathbf{M} = \begin{bmatrix} m - X_{\dot{u}} & 0 & 0 \\ 0 & m - Y_{\dot{v}} & m x_g - Y_{\dot{r}} \\ 0 & m x_g - N_{\dot{v}} & I_z - N_{\dot{r}} \end{bmatrix}$$

- For the Otter, with the payload this course uses, that matrix is

$$\mathbf{M} = \begin{bmatrix} 85.5000 & 0 & 0 \\ 0 & 162.5000 & 12.2500 \\ 0 & 12.2500 & 42.6515 \end{bmatrix} \quad \text{kg, kg·m, kg·m}^2$$

> **★ The surge row of $\mathbf{M}$ is already decoupled, and that is structural**
>
> The zeros in the first row and column are **exactly zero**, not small. They follow from **port–starboard symmetry**: a hull that is mirror-symmetric cannot produce a sway force or a yaw moment when it is accelerated purely forward. Everything below concerns $\mathbf{C}$ and $\mathbf{D}$, because $\mathbf{M}$ has already given the surge axis to itself.
>
> The off-diagonal $M_{26} = 12.25$ kg·m couples sway and yaw, and it is the reason a turn produces sway. It never touches surge.

- The matrix above is not transcribed. It is rebuilt from `otter.m` lines 55–120 and printed by

```
verify_constants
```

which also checks the sixteen coefficients this course quotes and reports `every quoted value agrees with otter.m to its printed precision`.

- The Coriolis matrix contributes to surge through the rigid-body and added-mass parts. Taking the first row of $\mathbf{C}_{RB}(\boldsymbol{\nu})\boldsymbol{\nu} + \mathbf{C}_A(\boldsymbol{\nu})\boldsymbol{\nu}$ and collecting terms gives the standard manoeuvring form:

$$\underbrace{(m - X_{\dot u})\,\dot u}_{\text{acceleration}} - \underbrace{(m - Y_{\dot v})\,vr}_{\text{Coriolis, sway}\times\text{yaw}} - \underbrace{(mx_g - Y_{\dot r})\,r^2}_{\text{centripetal}} = \underbrace{X}_{\text{thrust}} + \underbrace{X_u u}_{\text{damping, } X_u<0}$$

What is dropped, and why it is exact here

- Two terms stand between the full row and the scalar plant, and **both carry a factor of r**:

Term	Dropped because
$(m - Y_{\dot v})\,vr$	proportional to r
$(mx_g - Y_{\dot r})\,r^2$	proportional to r^2

- In this week's experiment r is not merely small. It is **identically zero**, and the reason is the actuator, not the hull:

$$\boldsymbol{\tau} = \mathbf{B}\mathbf{f}, \qquad \mathbf{B} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \\ y_p & -y_p \end{bmatrix}, \qquad \mathbf{f} = \begin{bmatrix} T \\ T \end{bmatrix} \implies \boldsymbol{\tau} = \begin{bmatrix} 2T \\ 0 \\ 0 \end{bmatrix}$$

- The speed controller splits its demand **equally** between the two propellers, so $N = y_p(T - T) = 0$ exactly. With $N = 0$ and $r(0) = 0$ the yaw equation gives $r \equiv 0$, and with $r \equiv 0$ the sway equation gives $v \equiv 0$.
- Therefore both discarded terms are **exactly zero for the whole run**, not small. This is the rare case where a 1-DOF reduction is not an approximation.

$$\boxed{(m - X_{\dot u})\,\dot u = X + X_u u} \quad \Longleftrightarrow \quad M_{11}\,\dot u = X + X_u u$$

$$M_{11} = (m + m_p) - X_{\dot u} = 85.50 \text{ kg}$$

> **i When the dropped terms come back**
>
> The moment a heading command is added, $r \neq 0$ and vr reappears in the surge equation — the vessel slows in a turn without any change in thrust. Week 1 measured exactly this: surge fell from 1.0286 to 1.0218 m/s in the turns, a loss of 0.7%, produced entirely by the term dropped above. Week 3 controls heading, Week 4 runs a guidance loop above it, and Week 7 closes surge and heading together.

- Taking Laplace transforms with $u(0) = 0$ gives a first-order lag:

$$\frac{u(s)}{X(s)} = \frac{K_u}{T_u s + 1}, \qquad K_u = \frac{1}{|X_u|}, \qquad T_u = \frac{M_{11}}{|X_u|}.$$

Symbol	Quantity	Value / source
M_{11}	surge mass including added mass	85.50 kg, `otter.m`
X_u	linear surge damping	-77.5544 N per m/s, $-24.4g/U_{\max}$
K_u	DC gain	0.012894 (m/s)/N
T_u	time constant	1.1025 s

> **★ The plant has no free integrator**
>
> There is no $1/s$ anywhere in $u(s)/X(s)$. The loop is therefore **type 0**, and every consequence of §2-2 follows from that single structural fact rather than from any numerical value.

2-2. Proportional control, and the error it cannot remove

- With $X = K_p(u_d - u)$ the closed loop is

$$\frac{u(s)}{u_d(s)} = \frac{K_p K_u}{T_u s + 1 + K_p K_u},$$

- and the final value theorem gives the steady state directly:

$$\boxed{\frac{u_{ss}}{u_d} = \frac{K_p K_u}{1 + K_p K_u} \quad \Longrightarrow \quad \frac{e_{ss}}{u_d} = \frac{1}{1 + K_p K_u}}$$

- The error is never zero for finite K_p. The physical reason is more useful than the algebraic one:

> **ℹ The error is what produces the force**
>
> Holding a steady speed requires a steady force, $X_{ss} = |X_u|\, u_{ss}$, because the damping never stops. A proportional controller produces force **only** from error. Removing the error would remove the force that sustains the speed, so the error is not a defect of the controller — it is the controller's only means of doing its job.

- Raising the gain shrinks the error but never removes it, and it does so at a cost. The gain also multiplies measurement noise and pushes the demanded force toward the saturation limits of §2-6.

2-3. Integral action, and the zero nobody placed

- Adding $K_i \int e\,\mathrm{d}t$ supplies the steady force without steady error. The closed loop becomes second order:

$$\frac{u(s)}{u_d(s)} = \frac{K_u\,(K_p s + K_i)}{T_u s^2 + (1 + K_u K_p) s + K_u K_i}.$$

- Matching the denominator to $s^2 + 2\zeta\omega_n s + \omega_n^2$ gives the design equations used in `W02_0_setup.m`:

$$\omega_n = \sqrt{\frac{K_u K_i}{T_u}}, \qquad \zeta = \frac{1 + K_u K_p}{2\sqrt{T_u K_u K_i}}.$$

- Inverting them for a specified pair:

$$K_i = \frac{\omega_n^2\, T_u}{K_u}, \qquad K_p = \frac{2\zeta\sqrt{T_u K_u K_i} - 1}{K_u}.$$

- For $\zeta = 0.7$ and $\omega_n = 1.5$ rad/s this gives $K_p = 102.00$ and $K_i = 192.38$.

> **✕ The tabulated overshoot for $\zeta = 0.7$ does not apply**
>
> The numerator of the closed loop is $K_u(K_p s + K_i)$, which is a **zero** at $s = -K_i/K_p = -1.886$. The standard relation $M_p = \exp\left(-\pi\zeta/\sqrt{1-\zeta^2}\right)$ is derived for a second-order system with **no** zero. A PI controller always adds one, and it always lands at $-K_i/K_p$, which for any sensible design sits close to the poles. The measured overshoot is 8.1% against the tabulated 4.6% — a factor of 1.8. The damping ratio was designed correctly; the prediction made from it was not.

2-4. Where the derivative term actually goes

- The derivative is taken on the **measurement**, not on the error:

$$X = K_p e + K_i \int e\,\mathrm{d}t \underbrace{-}_{\text{not a typo}} K_d \frac{Ns}{s+N}\,u.$$

> **★ Why that third sign is a minus while the other two are plus**
>
> The textbook PID is written $X = K_p e + K_i \int e\,\mathrm{d}t + K_d \dot{e}$ — three plus signs. The minus appears here because the last term is no longer built from e. Substituting $e = u_d - u$ into the derivative,
>
> $$\dot{e} = \dot{u}_d - \dot{u},$$
>
> and on a setpoint that is held constant between steps $\dot{u}_d = 0$, which leaves
>
> $$K_d \dot{e} = -K_d \dot{u}.$$
>
> So the minus **is** the plus of the textbook form, rewritten in terms of the measurement. Nothing about the controller changed; only which signal is differentiated. Writing $+K_d\dot{u}$ instead would apply damping with the wrong sign and drive the loop unstable, so the sign is worth checking rather than copying.
>
> The two forms differ in exactly one respect: at the instant of a setpoint step, $\dot{u}_d$ is an impulse, and the textbook form passes it to the actuator. That is the derivative kick the next row of the table refers to.

- Two reasons, and only the second is about this week.

Choice	Reason
on the measurement, not the error	a step in u_d differentiates to an impulse; the measurement never steps
filtered by $Ns/(s+N)$	pure differentiation has unbounded high-frequency gain and amplifies sensor noise without limit

The pseudo-derivative

- The second row deserves more than a line, because **no working PID contains a differentiator.** What it contains is an approximation to one, and that approximation has a name.

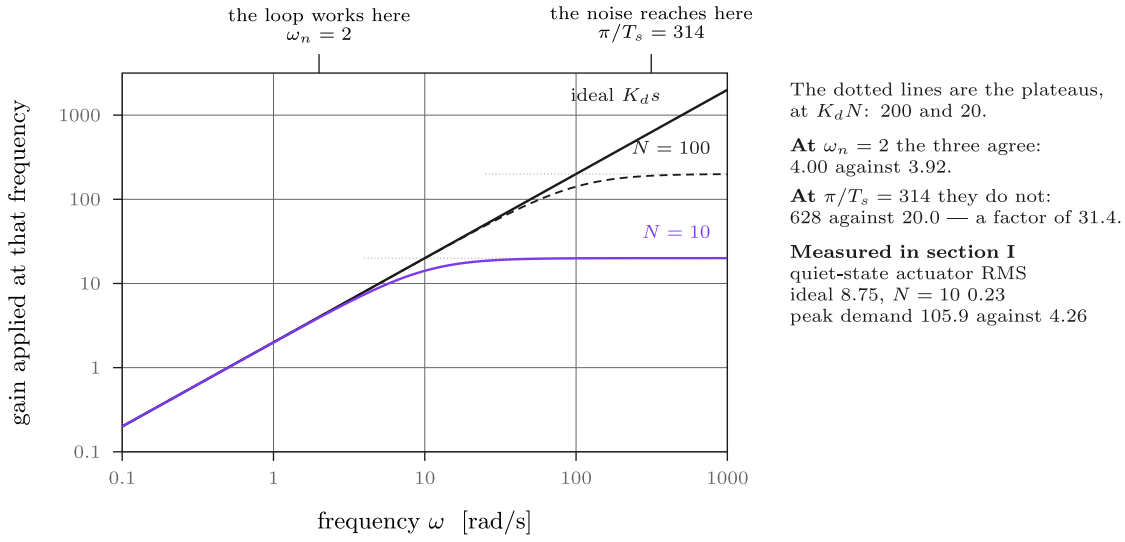
$$\underbrace{s}_{\text{ideal}} \quad \longrightarrow \quad \underbrace{\frac{Ns}{s+N}}_{\text{pseudo-derivative}}$$

- Compare the two by magnitude alone:

$$|j\omega| = \omega \qquad \textbf{against} \qquad \left|\frac{Nj\omega}{j\omega + N}\right| = \frac{N\omega}{\sqrt{\omega^2 + N^2}}$$

ω	ideal	pseudo
$\omega \ll N$	ω	$\approx \omega$ — the two agree
$\omega = N$	N	$N/\sqrt{2}$
$\omega \gg N$	$\omega \to \infty$	$\to N$, **flat**

- The table is easier to believe as a picture. The two operators are drawn below for $K_d = 2$, on axes where each gridline is a factor of ten.



Reading the figure

Element	Meaning
both axes	logarithmic. One gridline is a **factor of ten**, so a straight line of slope 1 means "gain proportional to frequency"
red	the ideal derivative, $\lvert K_d j\omega\rvert = K_d\omega$. A straight line that **never stops rising**
blue, $N = 10$	the pseudo-derivative. Identical to the red line at low frequency, then **flat at $K_dN = 20$**
amber, $N = 100$	the same shape with the ceiling ten times higher, at $K_dN = 200$
blue dot	the corner, at $\omega = N$ exactly. Below it the filter is invisible; above it the filter is everything
teal tick	where the closed loop actually works, $\omega_n = 2$ rad/s. **All three curves agree here**
violet tick	how fast the sensor can report, $\pi/T_s = 314$ rad/s. **The three curves differ by two orders of magnitude here**

What the figure says

One picture of why a derivative is dangerous and why one number fixes it. Across the bottom is frequency — slow signals on the left, fast ones on the right. Up the side is how much the operator multiplies a signal at that frequency.

The red line is the ideal derivative, and it **never stops climbing.** That is the whole problem, and it is worth stating carefully: differentiation is not inaccurate. Its gain is simply unbounded, so whatever is fastest in the measurement gets amplified most — and sensor noise is always the fastest thing present.

Now compare the red and blue lines at the two ticks. At the teal tick, the frequency the controller is actually designed to work at, the ideal derivative gives $K_d\omega_n = 4.00$ and the $N = 10$ filter gives 3.92 — two per cent apart. At the

violet tick, the fastest thing the sensor can report, the ideal gives 628 and the filter gives 20.0 — a factor of **31.4** apart.

The two operators are the same where the work is done and utterly different where the noise is. That is the entire argument for filtering.

Look at what the blue curve does and does not do. It lies exactly on the red one up to $\omega = N$ and then stops climbing. It does not make the derivative *better* below N; it only refuses to keep amplifying above it. So choosing N just above the closed-loop bandwidth costs nothing in the response and removes everything above it.

That also settles why a large N is not a cautious default. Moving from $N = 10$ to $N = 100$ lifts the whole right-hand end by a factor of ten while changing nothing near ω_n — more noise, no more control. Section I measures exactly that: the response barely moves and the actuator's quiet-state RMS grows from 0.23 to 1.68.

> **★ Differentiation multiplies every component by its own frequency**
>
> That is the whole problem in one sentence. The fastest thing in any real measurement is the sensor noise, so differentiation seeks out the least meaningful part of the signal and multiplies it by the largest number in the problem. The pseudo-derivative is the same operator with its gain capped at N, and N is therefore the knob that decides **how much of the noise reaches the actuator**.

- Written as a state, the pseudo-derivative is a first-order lag on the signal, subtracted from it:

$$\frac{Ns}{s+N} = N\left(1 - \frac{N}{s+N}\right) \qquad \Longleftrightarrow \qquad \dot{x}_f = N\,(y - x_f), \quad d = N\,(y - x_f)$$

- So it costs one state and no differentiation at all. **A low-pass filter and a subtraction are doing the work of a derivative**, which is why it is implementable in fixed point on a microcontroller and why an ideal derivative is not.

Choosing N	What happens
N too small	the filter lags, the derivative arrives late, and the damping it was added for is lost
N too large	the response stops improving — the pole is far outside the loop bandwidth — while the noise keeps growing

- There is a common belief that large N is the safe default because it is "closer to a true derivative". Section I measures it and finds the opposite: above a certain N the response is unchanged to within 0.3 points of overshoot while the actuator noise grows by a factor of three.
- The rule that follows: **place N just above the closed-loop bandwidth, and no higher.**

> **ℹ The name**
>
> Textbooks call $Ns/(s+N)$ the *filtered*, *approximate*, *practical* or *pseudo*-derivative interchangeably. Åström and Hägglund parameterise it as $T_d s/(1 + (T_d/N)s)$, where $T_d = K_d/K_p$ and $N \in [8, 20]$ is dimensionless. That N and the N used here are related by $N_{\text{here}} = N_{\text{Åström}}/T_d$, so the two conventions must never be mixed in one derivation.

- Now substitute into the equation of motion. Ignoring the filter, $-K_d\dot{u}$ moves to the left-hand side:

$$(M_{11} + K_d)\dot{u} = X_{\text{PI}} + X_u u.$$

★ On a velocity loop the derivative term is a mass, not a damper

The controlled variable is a velocity, so its derivative is an **acceleration**, and K_d enters the equation of motion in exactly the place occupied by the mass. The effective time constant becomes $\tau_{\text{eff}} = T_u + K_u K_d$, so

$$\omega_n = \sqrt{\frac{K_u K_i}{T_u + K_u K_d}} \downarrow, \qquad \zeta = \frac{1 + K_u K_p}{2\sqrt{(T_u + K_u K_d) K_u K_i}} \downarrow.$$

Adding derivative action to this loop makes it **less** damped, not more. Section E measures it.

- This is a property of the **axis**, not of PID. Week 3 controls a heading, whose derivative is a rate rather than an acceleration, and there the same term supplies genuine damping. The lesson is to substitute the control law into the equation of motion before assuming what a term does.

2-5. Allocation, in its simplest form

- The controller demands a force. The propellers accept a shaft speed. Inverting $T = k\,n|n|$ with the demand split equally:

$$T = \frac{X_{\text{cmd}}}{2}, \qquad n = \text{sign}(T)\sqrt{\frac{|T|}{k}}, \qquad k = \begin{cases} k_{\text{pos}}, & T \geq 0 \\ k_{\text{neg}}, & T < 0 \end{cases}$$

- then saturate n and compute what the propellers **actually** deliver:

$$X_{\text{sat}} = 2\,k\,n_{\text{sat}}|n_{\text{sat}}|.$$

- The equal split is forced: a pure surge demand contains nothing that distinguishes the two propellers. Week 5 treats the case where the demand does distinguish them and the split is no longer obvious.

⚠ X_{sat} must leave the allocation block

Without it the controller has no way of knowing that the actuator has stopped following the demand. Every anti-windup scheme in §2-6 is built on the difference $X_{\text{cmd}} - X_{\text{sat}}$, and a block that does not report what it delivered makes all of them impossible.

2-6. Windup is a saturation problem

Why anti-windup is needed, before any equation

The integrator of §2-4 is a **memory of past error**. Every second the vessel runs slow, the integrator remembers it, and that memory is what eventually removes the offset a proportional term cannot. This is the whole reason the term exists, and it works because the memory is always *acted upon*: the integrator asks for more force, the force arrives, the error shrinks, and the memory stops growing.

Windup is what happens when that last sentence stops being true. Follow one run:

Section F runs exactly this on the Otter, commanding $u_d = 3.5$ m/s when $u_{\max} = 3.0864$ m/s, and every number below is measured there:

Time	What is commanded	What the vessel does	What the integrator does
$t = 5$ s	3.5 m/s — the propellers **cannot** reach it	accelerates to 3.09 m/s and stays there	the error never reaches zero, so the memory **keeps growing**
5 to 40 s	the same impossible speed	nothing further — it is already flat out	climbs to 3438 N, against a largest *deliverable* force of 239 N
$t = 40$ s	1.5 m/s — now reachable	still flat out, and now wrongly so	begins to unwind, from 14.4 times the useful value
40 to 54 s	the same reachable speed	**holds 3.09 m/s for another ten seconds**, then overshoots	every stored newton must be integrated away before the demand re-enters the attainable set

- The vessel spends **fourteen seconds ignoring a command it could have obeyed at once** — a 13.98 s recovery against 3.40 s with clamping — and nothing is broken. Every block did exactly what it was built to do.
- The trouble is that between $t = 5$ and $t = 40$ s the integrator was recording an error it had no power to remove. Its memory is a record of a negotiation the actuator had already lost, and on any measure that matters it is **remembering something that never happened**.
- Anti-windup is therefore not a repair to the integrator. It is a way of **telling the integrator that the loop is open**, so that it stops recording while its recording cannot mean anything. That is the single idea, and everything below is two ways of saying it in equations.

> **Why this cannot be tuned away**
>
> Lowering K_i makes the integrator wind up more slowly, and also makes it remove the offset more slowly — the two are the same coefficient. No value of K_i separates them, because the problem is not the size of the gain but the fact that the loop is open while the gain is applied. A structural fault needs a structural fix, which is why a *scheme* is added rather than a number changed.

The saturation that causes it

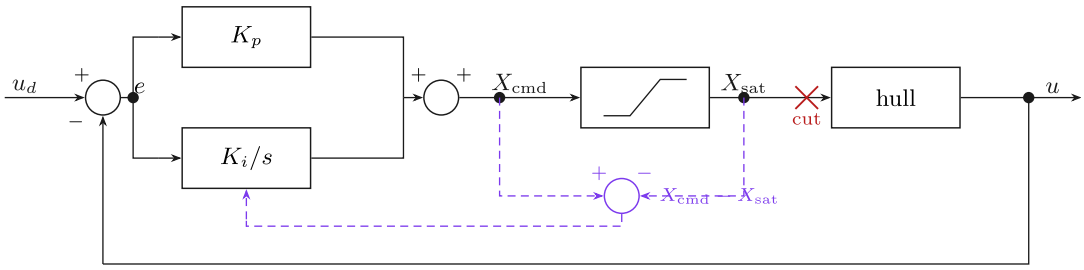
- The attainable set of §A1-6 caps the surge force at $X_{\max} = 24.4g = 239.364$ N. Combined with $X_u = -24.4g/U_{\max}$ this produces an exact and rather elegant limit:

$$u_{\max} = \frac{X_{\max}}{|X_u|} = \frac{24.4g}{24.4g/U_{\max}} = U_{\max} = 3.0864 \ \text{m/s}.$$

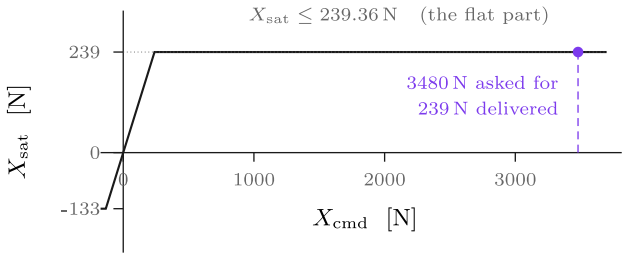
- The saturation limit and the damping coefficient are not independent numbers in `otter.m`. They are chosen so that full ahead delivers exactly the design speed of six knots. Astern, $X_{\min} = -13.6g$ gives $u_{\min} = -1.7203$ m/s.
- Ask for more than $u_{\max}$ and the error cannot be driven to zero. The integrator, which knows nothing of this, keeps integrating.

Stage	What happens
demand exceeds the limit	X_{sat} stops following X_{cmd}; the loop is open
the integrator continues	its state grows without bound while the error stays positive
the setpoint returns to a reachable value	the error changes sign, but the stored charge must be integrated back out before X_{cmd} re-enters the attainable set
meanwhile	the vessel does not respond at all

(a) where the loop breaks



(b) how far outside the limit the demand went



Reading the figure

Element	Meaning
left, the block diagram	the ordinary PI loop, with the saturation drawn as its own block between the controller and the hull
red cross	where the loop is broken while the actuator is on its limit
violet dashed path	the signal $X_{cmd} - X_{sat}$, which **both** anti-windup schemes add and neither can do without
right, the amber curve	the saturation block itself, plotted to scale: what comes out against what went in
right, red marker	the largest demand measured in section F, placed on that curve

What the figure says

The left panel says *why* windup happens. The right panel says *how badly* it happened on this vessel.

Start on the right. The amber curve is a straight line only between -133.42 and $+239.36$ N. Outside that band it is flat — and the controller's demand reached 3480 N, **fourteen and a half times** the largest force the propellers can produce. Everything to the right of that corner was asked for and not delivered.

The left panel names the consequence. While the actuator is flat, X_{sat} no longer depends on X_{cmd}: the controller's output has no effect on the hull, and therefore none on the error. **The loop is open.** An integrator driven by the error of an open loop cannot converge, because nothing it does can change that error.

Notice where the red cross is drawn — between the saturation and the hull, not inside the integrator. That placement is the argument. The integrator is behaving exactly as designed; the fault is upstream of it, in a block that has no gain left to give. Calling this an "integrator problem" points at the wrong component and leads to the wrong fix.

The violet path is what both cures are built from. It carries $X_{cmd} - X_{sat}$, and that difference is **identically zero whenever the actuator is following its command.** Clamping and back-calculation use it differently, but neither can act on a loop that is not saturated — which is why anti-windup can be switched on permanently and never has to be scheduled.

The principle, in one line

- Write the controller output and what the actuator actually delivers as two separate signals:

$$X_{cmd} = K_p e + I, \qquad X_{sat} = \mathrm{sat}(X_{cmd}), \qquad \dot{I} = K_i e.$$

- Everything goes wrong at the third equation. $\dot{I}$ is written in terms of e, and e is the error of a loop that is **no longer closed**. Both remedies do the same thing: they make $\dot{I}$ depend on the saturation as well.

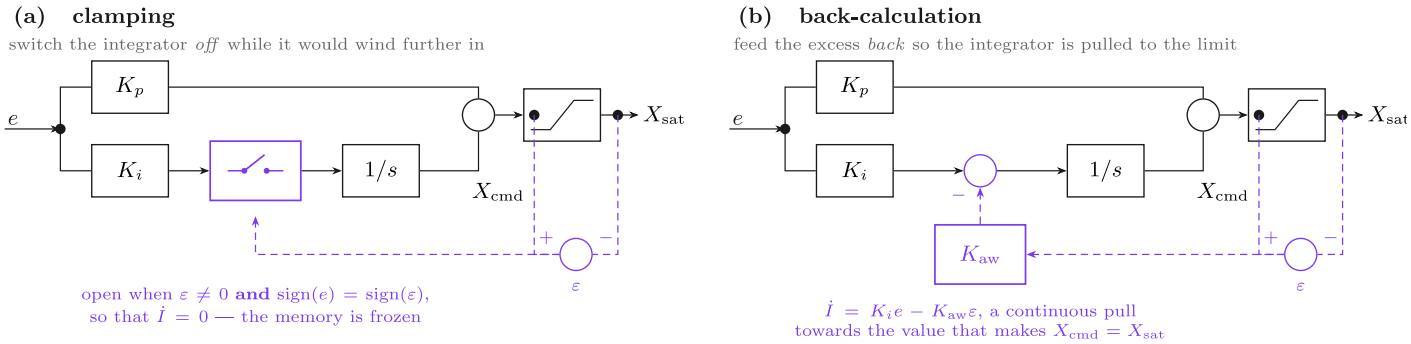
$$\dot{I} = K_i e \; - \; \underbrace{f\left(\overbrace{X_{\text{cmd}} - X_{\text{sat}}}^{\varepsilon},\; e\right)}_{\text{zero whenever the actuator is following}}$$

> **★ What f is, and why it is written as a letter**
>
> f is **not a specific function.** It is a placeholder for "whatever the anti-windup scheme subtracts", and the equation above is the *shape* both schemes share rather than either one of them. Writing it this way makes the shared requirement visible before the two schemes disagree about how to meet it.
>
> The one requirement is $f(0, e) = 0$: **no excess, no correction.** Since $\varepsilon \equiv X_{\text{cmd}} - X_{\text{sat}}$ is identically zero whenever the actuator is following its command, this guarantees that an unsaturated loop is untouched by any scheme meeting it — which is what makes anti-windup safe to leave switched on permanently, with nothing to schedule.
>
> The two remedies below are two choices of f, and the table names them:
>
> | Scheme | $f(\varepsilon, e)$ | $f(0, e)$ |
> |---|---|---|
> | back-calculation | $K_{\text{aw}}\,\varepsilon$ — proportional to the excess | 0 ✓ |
> | clamping | $K_i e$ when $\varepsilon \neq 0$ **and** $\text{sign}(e) = \text{sign}(\varepsilon)$; otherwise 0 | 0 ✓ |
>
> Clamping's f needs e as well as ε, which is why f carries two arguments. Subtracting exactly $K_i e$ leaves $\dot{I} = 0$: that is what "freeze the integrator" means algebraically. Back-calculation's f ignores e entirely and depends only on how far outside the limit the demand went.

Two remedies

Two anti-windup schemes, and the one place they differ

Everything outside the violet block is identical, including the excess $\varepsilon = X_{\text{cmd}} - X_{\text{sat}}$ that both schemes measure. The schemes disagree only about what to do with it at the integrator's input.



Reading the figure

Element	Meaning
everything in black	identical in both schemes — the same K_p, the same K_i, the same integrator, the same saturation
violet dashed path	the excess $\varepsilon = X_{\text{cmd}} - X_{\text{sat}}$, measured the same way on both sides
(a) violet block	a switch that **opens**, cutting the integrator's input to zero
(b) violet block	a gain K_{aw} whose output is **subtracted** from the integrator's input

Both schemes tap the same two points — just before the saturation and just after it — and subtract them. When the actuator is following its command those two points carry the same signal, so $\varepsilon = 0$ and neither violet block does anything at all. That is the shared property, and it is visible as the fact that the violet ink only ever *adds* to the black diagram.

Where they part is a single block. Clamping opens a switch, so the integrator stops dead and keeps whatever it was holding. Back-calculation subtracts a number proportional to the excess, so the integrator keeps moving but is pulled steadily toward the value that would make the demand equal the limit. **One stops the memory; the other steers it.**

That difference is why raising K_{aw} never turns back-calculation into clamping: a switch and a pull are not two settings of one thing.

Clamping, or conditional integration — freeze the integrator while the actuator is saturated *and* the error would drive it further in:

$$\dot{I} = \begin{cases} 0, & |X_{\text{cmd}} - X_{\text{sat}}| > 0 \ \text{ and } \ \operatorname{sign}(e) = \operatorname{sign}(X_{\text{cmd}} - X_{\text{sat}}) \\ K_i e, & \text{otherwise} \end{cases}$$

Back-calculation — feed the excess back into the integrator through a gain:

$$\dot{I} = K_i e - K_{\text{aw}}\,(X_{\text{cmd}} - X_{\text{sat}}), \qquad K_{\text{aw}} = \frac{1}{T_u}.$$

- Back-calculation has a fixed point worth naming. While the actuator is saturated, $\dot{I} \to 0$ requires

$$I \to \frac{K_i}{K_{\text{aw}}} e + X_{\text{sat}} - K_p e,$$

- which is the integrator value that would make the demand equal to what the actuator can give. The scheme does not merely stop the integrator; it **steers it to the boundary**.

	Clamping	Back-calculation
mechanism	logical, the integrator is switched off	continuous, the integrator is pulled to the boundary
tuning	none	one gain, K_{aw}
behaviour at the boundary	discontinuous	smooth
where the integrator ends up	wherever it was when saturation began	at the value that makes $X_{\text{cmd}} = X_{\text{sat}}$
large-gain limit	—	**not** clamping; §H measures the difference

> **Neither is a fix for the integrator**
>
> Both schemes exist because the loop was opened by the actuator. The correct engineering response to persistent saturation is a reference the vessel can actually follow — which is what Week 7's reference model provides. Anti-windup limits the damage; it does not make an unreachable setpoint reachable.

2-7. The same controller, written twice

- Simulink ships a **PID Controller** block that contains everything §2-3 to §2-6 describes: three terms, a filtered derivative, an output limit, and the same two anti-windup schemes as dialog options.
- Both are in the model, and `pid_mode` chooses between them.

`pid_mode`	Path
0	the hand-built controller — nine blocks
1	the library block — one block

- The reason for having both is not indecision. A learner who has only used the block does not know what is inside it; a learner who has only built it by hand does not know the block exists, and will rebuild it on every project.
- **Two differences are real**, and section G measures them rather than hiding them.

Difference	Consequence
the library block differentiates its **input**, which here is the error	with $K_d = 0$ the two agree exactly; with $K_d > 0$ they do not
the library block saturates its **own** output at $[X_{\min}, X_{\max}]$	the same limits the allocation imposes, so the two saturations coincide

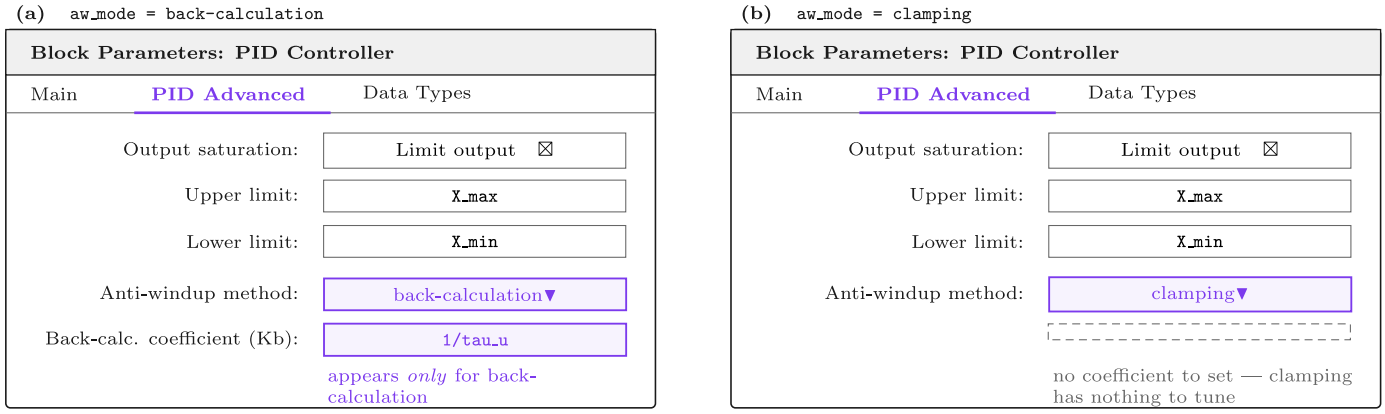
- The first difference has a name and a remedy. Differentiating the error puts every setpoint step through the derivative; differentiating the measurement does not. Simulink's **two-degree-of-freedom** PID block exists precisely to let the two be weighted separately.

Selecting the anti-windup scheme in the block

- Both schemes of §2-6 are already inside the library block. Neither has to be built: they are chosen from one menu, on the **PID Advanced** tab.
- The menu is only active once **Limit output** is ticked on that tab. This is not a quirk — a controller with no saturation cannot wind up, so there is nothing for the scheme to act on, exactly as the $f(0, e) = 0$ requirement of §2-6 said.

Where the two schemes are selected in the PID Controller block

A schematic of the dialog, not a screenshot. Both schemes live on the same tab, chosen from one menu; picking *back-calculation* is what makes the K_b field appear. Everything else on the tab is identical.



Reading the figure

Element	Meaning
PID Advanced tab	where all four fields live; nothing on the **Main** tab is involved
Limit output	must be ticked, or the anti-windup menu stays greyed out
Anti-windup method	the one menu — `none`, `back-calculation`, `clamping`
(a) K_b field	appears only for back-calculation, and is the K_{aw} of §2-6 under Simulink's name
(b) dashed box	the same place in the dialog with clamping selected — the field is simply not there

The dialog makes the structural difference of §2-6 visible before anything is run: back-calculation has a number to choose and clamping has none. A scheme that steers the integrator needs to be told how hard to pull; a scheme that switches it off does not.

Is it the same algorithm?

Yes, and the model measures it rather than asserting it. Section H builds back-calculation **by hand** from the equation of §2-6 and runs it beside the library block on the same plant, with the same gains and the same limit:

Row	Anti-windup	Recovery [s]	y at $t = 20$ s
3	back-calculation, **library block**	2.645	0.5003
4	back-calculation, **written by hand**	2.645	0.5003

- The two rows agree to $0.00\text{e}{+00}$ — not closely, but **bit for bit**. The equation printed in §2-6 is therefore a correct description of what the block computes, and not merely a scheme with the same name.
- Section G repeats the test on the whole controller rather than the anti-windup path alone, and with $K_d = 0$ the two implementations agree to 2.2×10^{-16} m/s across the entire run. The only place they part company is the derivative input, which is the first row of the table above and has nothing to do with anti-windup.

> **Why this test is worth running at all**
>
> A library block is a claim in a manual. Building the same thing from the equations and getting the same numbers turns that claim into something checked. It also settles the more useful question in the other direction: since the hand-built path matches, the equations of §2-6 can be trusted on a platform that has no PID block at all.

Part 2 · Laboratory

A. Setting up (10 min)

```
cd GradCourse/lectures/W02_simulink
W02_0_setup
```

Expected output:

```
  W02 setup complete
    plant           T_u = 1.1025 s,  K_u = 0.012894 (m/s)/N
    actuator        X in [-133.42, 239.36] N  ->  u_ss in [-1.7203, 3.0864] m/s
    controller      Kp = 102, Ki = 192.38, Kd = 0, Nf = 20
    closed loop     wn = 1.5000 rad/s,  zeta = 0.7000
    anti-windup     back-calculation
    reference       1.5 -> 1.5 m/s at t = 5 s
    simulation      30 s at h = 0.02 s
```

- To restore a model that has been broken: `W02_1_build_surge_control`.

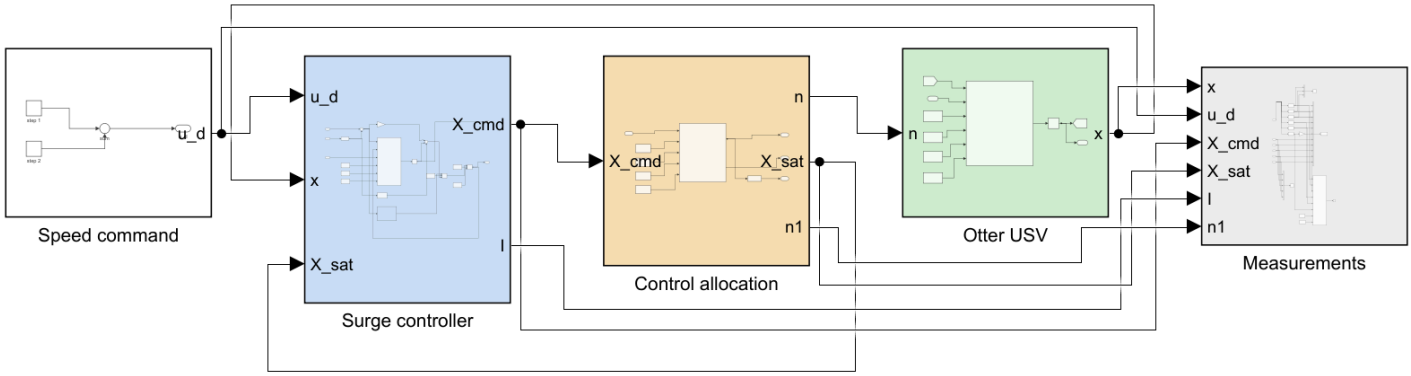
The files of this week, in the order the sections use them

Opening the folder shows about fifteen files. Only the ones in this table are ever run by hand.

Order	File	Section	What it produces
0	`W02_0_setup.m`	A	the base workspace. **The only file to edit this week**
1	`W02_1_build_surge_control.m`	B	`W02_surge_control.slx` and its block diagram
C	`W02_C_identify_plant.m`	C	`img/W02_result_openloop.png`
D	`W02_D_proportional_only.m`	D	`img/W02_result_P.png`
E	`W02_E_integral_and_derivative.m`	E	`img/W02_result_PI.png`, `img/W02_result_D.png`
F	`W02_F_windup.m`	F	`img/W02_result_windup.png`
G	`W02_G_block_vs_handbuilt.m`	G	`img/W02_result_block.png`
H	`W02_H_build_antiwindup.m` then `W02_H_antiwindup_run.m`	H	`img/W02_result_aw_principle.png`
I	`W02_I_build_pseudo_derivative.m` then `W02_I_pseudo_derivative_run.m`	I	`img/W02_result_pd.png`

- **One script per section.** Running a section leaves exactly the numbers and the figures that section discusses, so a class can work through the week a page at a time.
- Sections H and I have a **build** script and a **run** script, because each adds a model of its own. Run the build once, then the run script as often as needed.
- The remaining files — `W02_vars.m`, `W02_cols.m`, `W02_plot.m`, `W02_aw_plot.m`, `W02_pd_plot.m`, `W02_animate.m` — are called **by** the section scripts and by the model. They are never run by hand.

B. Reading the model (15 min)




```
WEEK 2  -  SURGE SPEED CONTROL

The first closed loop of the course. The surge axis is a first-order,
TYPE 0 plant:  u/X = K_u / (T_u s + 1),  K_u = 0.012894, T_u = 1.1025 s.

FOUR THINGS TO TRY

1  IDENTIFY THE PLANT.  loop_closed = 0, X_open = 100. Read the settled u
   and the 63 per cent time. They are K_u X_open and T_u.

2  PROPORTIONAL ONLY.  Ki = 0, Kd = 0. A type 0 plant under proportional
   control ALWAYS leaves a steady-state error:

       u_ss / u_d = K_p K_u / (1 + K_p K_u).

   Kp = 100 leaves 44 per cent. Kp = 2000 leaves 3.7 per cent and still
   does not arrive. Raising the gain does not remove the error.

3  ADD THE INTEGRATOR.  Ki = 192.38 with Kp = 102 places the closed-loop
   poles at zeta = 0.7, wn = 1.5 rad/s. The error goes to zero exactly.
   The overshoot does NOT match the textbook figure for zeta = 0.7,
   because a PI controller also adds a ZERO at s = -Ki/Kp.

4  SATURATE IT.  u_d1 = 3.5 m/s is not reachable: full ahead gives
   exactly 3.0864 m/s. Set t_dn = 40, u_d2 = 1.5, T_final = 90 and
   compare aw_mode = 0, 1, 2.

X_sat is the second feedback path, and it is the one that gets missed.
Without knowing what the propellers ACTUALLY delivered, no anti-windup
scheme can tell that the loop has been opened by the actuator.
```

Reading the figure

Element	Meaning
`Speed command` (white)	two steps summed, so one model covers a single step and the up-then-down profile of §F
`Surge controller` (blue)	the PID, the anti-windup selector, and the open/closed switch
`Control allocation` (sand)	$X_{cmd} \rightarrow n$, and back to X_{sat} after saturation
`Otter USV` (green)	`otter.m`, called unchanged
`Measurements` (grey)	selectors, scope, workspace log and live view

- The chain of Week 1 is now complete except for the reference model: **command → controller → allocation → plant → measurement**, in that order, left to right.
- **Two signals travel backwards, and only two.**

Signal	From	To	Why
`x`	plant	controller	the measured speed. The whole twelve-state vector is sent and the controller selects u inside
`X_sat`	allocation	controller	what the propellers **actually** delivered

> ★ `X_sat` **is the feedback path that gets missed**
>
> Without knowing what the actuator delivered, the controller has no way of telling that the demand was not followed. Every anti-windup scheme in §2-6 is built on the difference $X_{\text{cmd}} - X_{\text{sat}}$, so an allocation block that does not report what it produced makes all of them impossible.

Inside `Surge controller`

- Opening it shows the three terms, the anti-windup block that decides what reaches the integrator, and one switch:

Block	Meaning
`Kp`	the proportional term
`anti-windup`	`aw_mode` selects one of the three schemes of §2-6
`I state`	the integrator, and the only state in the controller
`D filter`	$K_d N s/(s+N)$ acting on the **measurement**, not on the error
`open or closed`	`loop_closed = 0` applies `X_open` instead of the controller

- The switch is not decoration. `loop_closed = 0` turns this model into the open-loop rig of §C, so the plant that is identified is provably the plant the controller then drives.

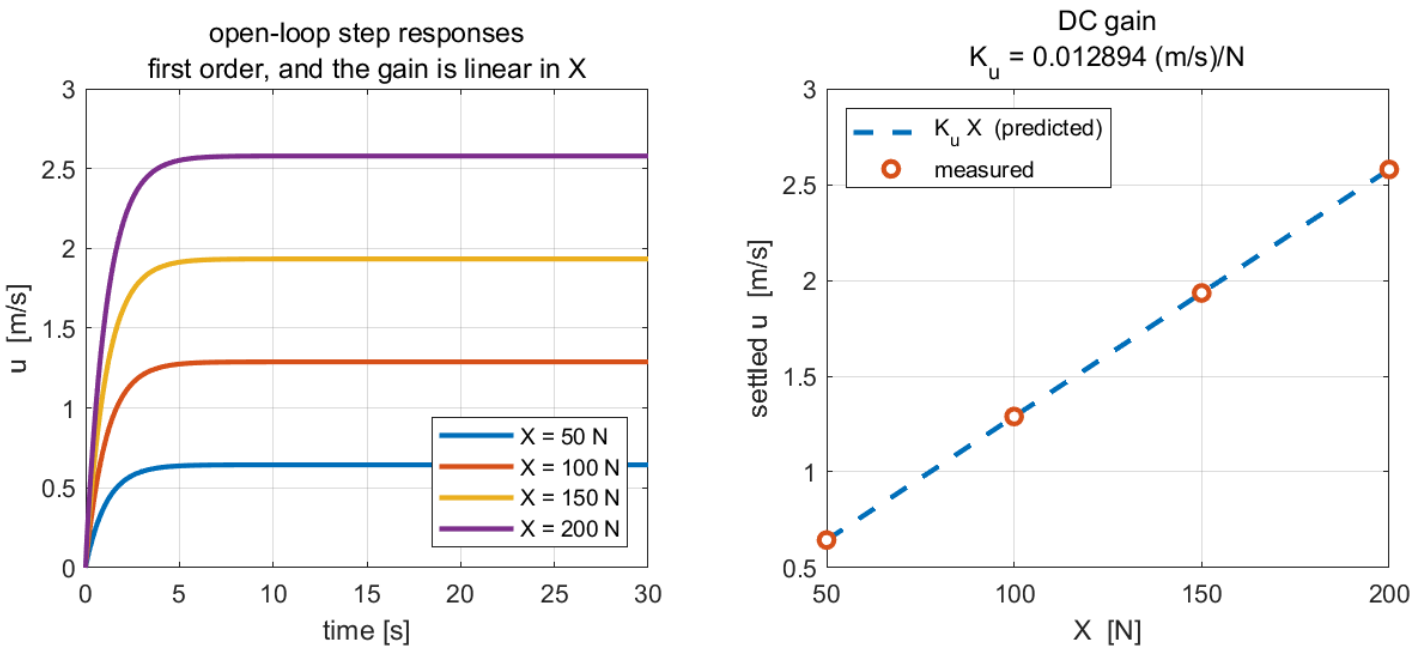
C. Identifying the plant from the plant (15 min)

> ✎ **To produce every figure in this section**
>
script	`W02_C_identify_plant.m`
model	`W02_surge_control.slx`
figure	`img/W02_result_openloop.png`

```
W02_0_setup                    % once per session
W02_C_identify_plant
```

Opening the model and pressing **Run** produces the same figure: the scopes update as it runs and `StopFcn` draws the summary at the end.

X [N]	measured u_{ss} [m/s]	$K_u X$ [m/s]	measured τ [s]	error [%]
50	0.6447	0.6447	1.1200	−0.00
100	1.2894	1.2894	1.1200	−0.00
150	1.9341	1.9341	1.1200	−0.00
200	2.5788	2.5788	1.1200	−0.00



Reading the figure

Element	Meaning
left panel	four open-loop step responses, one per applied force
right panel, dashed	the predicted DC gain line $u = K_u X$
right panel, circles	the settled speed measured from each of the four runs

- The DC gain is exact to four decimals across the whole range. The plant really is $u = K_u X$ in steady state.
- The time constant is **not** exact: 1.1200 s measured against 1.1025 s predicted, an excess of 1.59%. The residue is the surge-pitch coupling retained by the twelve-state plant and discarded by the scalar model. It is reported rather than absorbed.

What the figure says

There is no controller in this figure. The loop is open, a constant force is applied directly, and the vessel is asked what it does about it. The left panel is the answer in time; the right panel reduces each of those runs to the one number that describes where it ended up.

Two things have to be true before any of this week's design can proceed, and each panel checks one of them.

The four curves in the left panel have the **same shape** and differ only in height. Each rises smoothly, without overshoot or oscillation, and is within 2% of its final value by about 4.5 s. That shape is the signature of a first-order lag, $T_u \dot{u} + u = K_u X$ — one time constant and nothing else.

The right panel is a **straight line through the origin**. Doubling the force from 100 to 200 N doubles the settled speed from 1.2894 to 2.5788 m/s exactly, and all four measured points lie on the line to four decimals. That straightness is what says the plant is linear.

Neither property was guaranteed, which is why both were measured. The vessel underneath is a twelve-state nonlinear model, and it would have been entirely possible for it to curve.

The two numbers this figure produces — $K_u = 0.012894$ (m/s)/N and $T_u \approx 1.12$ s — are what every gain in sections D to F is computed from. Measuring them from the plant rather than reading them off a datasheet is what makes the later predictions checkable rather than merely plausible.

The line does end, though. At $X = 239.36$ N the propellers saturate, and the speed ceiling that follows is the one number no controller this week can argue with.

★ $u_{\max} = U_{\max}$ is exact, and it is not a coincidence

$X_{\max} = 2k_{\text{pos}}n_{\max}^2 = 24.4g$ by construction of $n_{\max}$, and $X_u = -24.4g/U_{\max}$ by construction of the damping. The two $24.4g$ cancel, leaving $u_{\max} = U_{\max} = 3.0864$ m/s exactly. No controller of any structure can ask for more.

D. Proportional only (15 min)

✎ To produce every figure in this section

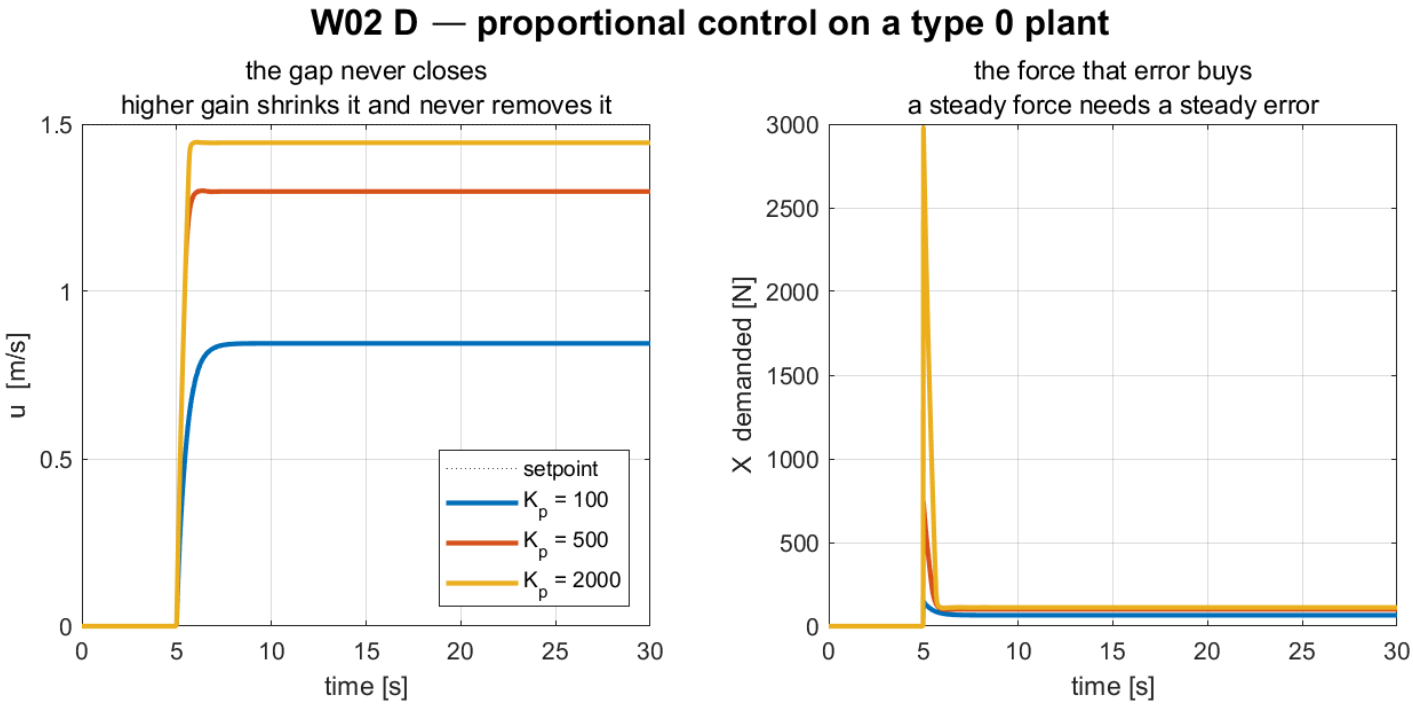
script	`W02_D_proportional_only.m`
model	`W02_surge_control.slx`
figure	`img/W02_result_P.png`

```
W02_0_setup                    % once per session
W02_D_proportional_only
```

Opening the model and pressing **Run** produces the same figure: the scopes update as it runs and `StopFcn` draws the summary at the end.

Measured with $u_d = 1.5$ m/s, $K_i = K_d = 0$:

K_p	K_pK_u	predicted u_{ss}	measured u_{ss}	error [%]	X_{ss} [N]	peak X_{cmd} [N]
100	1.2894	0.8448	0.8448	43.68	65.519	149.3
500	6.4471	1.2986	1.2986	13.43	100.711	744.8
2000	25.7883	1.4440	1.4440	3.73	111.989	2979.2



Reading the figure

Element	Meaning
left panel	three step responses; none reaches the dashed setpoint
right panel	the force each controller demanded, transient and steady

- Prediction and measurement agree to four decimal places, because both are consequences of the same structural fact and neither involves an approximation.
- A twentyfold increase in gain buys a reduction from 44% to 3.7%. The right panel shows why the error cannot vanish: the steady force needed is 111.99 N at the highest gain, and a proportional controller can only produce it from a non-zero error.

What the figure says

One setpoint of 1.5 m/s, three proportional controllers. The left panel is how close each got; the right panel is what each had to demand to get there.

None of them arrives. Raising K_p from 100 to 2000 moves the settled speed from 0.8448 to 1.4440 m/s — closer at every step and **short at every step**. The remaining gap goes 44%, 13%, 3.7%: each twentyfold increase in gain buys about one decimal place, and no increase buys the last one.

That is not a tuning failure, and no value of K_p would fix it. The plant has no free integrator, so the loop is **type 0**, and the final-value theorem gives

$$\frac{u_{ss}}{u_d} = \frac{K_p K_u}{1 + K_p K_u},$$

a ratio that approaches one and never reaches it. The physical reason is simpler than the algebra: holding a speed requires a permanent force to balance the drag, and a proportional controller manufactures force only out of error. **So the error is what pays for the force, and it cannot be zero.**

The right panel makes that visible. All three demands settle at roughly 65–112 N and stay there — and that plateau *is* the drag at whatever speed each vessel reached. Compare the same panel in Week 3, where the demand returns to **zero** because a vessel that has stopped turning needs no moment to hold its heading. The structural difference between the two axes is readable in this one plot.

What the gain does buy is paid for in actuator authority, and both run out together. At $K_p = 2000$ the peak demand is 2979.2 N — **twelve times** what the propellers can deliver, since $X_{\max} = 239.36$ N — so the response drawn for that run is already partly fictional.

Section E adds the integral term, which supplies the steady force from a **zero** error and settles the question rather than shrinking it.

E. Integral and derivative (20 min)

✎ To produce every figure in this section

script	`W02_E_integral_and_derivative.m`
model	`W02_surge_control.slx`
figure	`img/W02_result_PI.png and img/W02_result_D.png`

```
W02_0_setup                    % once per session
W02_E_integral_and_derivative
```

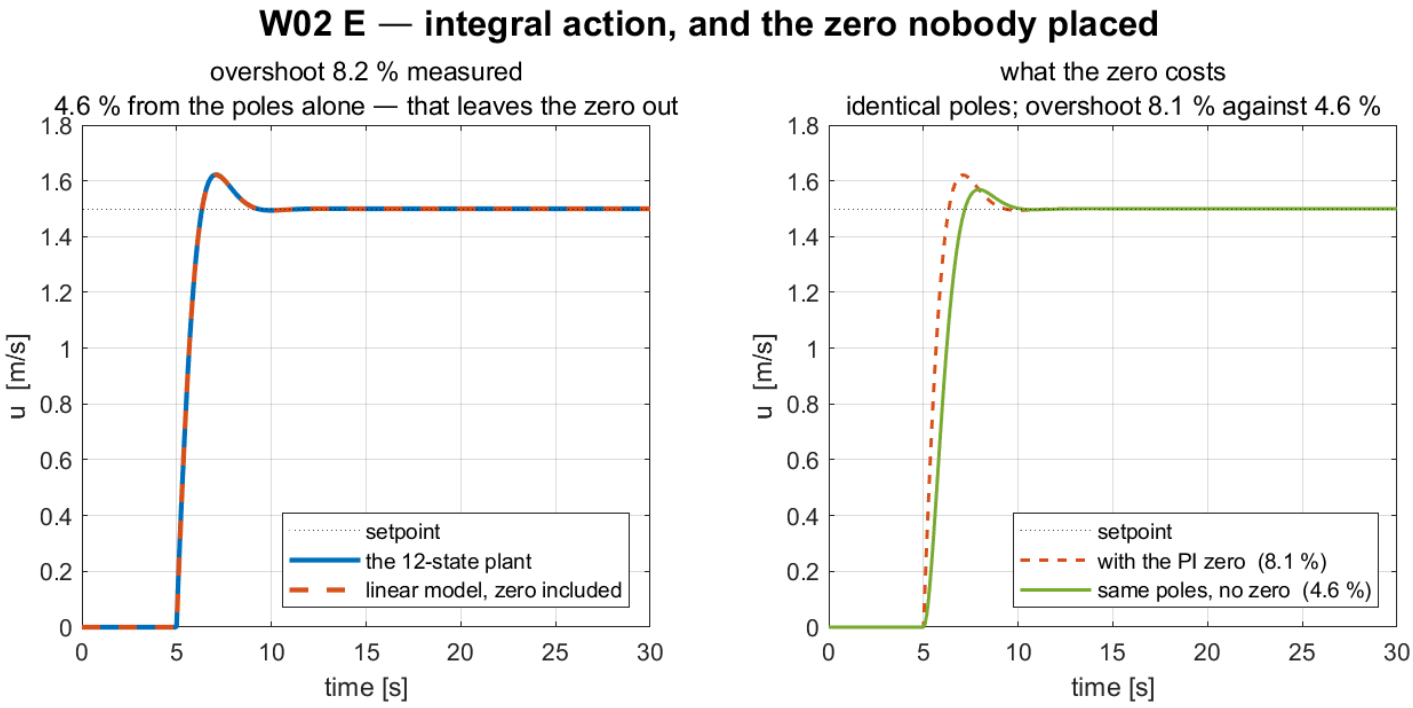
Opening the model and pressing **Run** produces the same figure: the scopes update as it runs and `StopFcn` draws the summary at the end.

PI

Designed for $\zeta = 0.7$, $\omega_n = 1.5$ rad/s, giving $K_p = 102.00$ and $K_i = 192.38$, with the zero at $s = -1.8861$ and the poles at $-1.050 \pm 1.071\mathrm{j}$.

Source	overshoot [%]	t_s (2%) [s]
poles only, $\exp\left(-\pi\zeta/\sqrt{1-\zeta^2}\right)$	4.60	3.81
linear model **including** the PI zero	8.07	3.47
the twelve-state plant	8.15	3.46

- Steady-state error at $t = 30$ s: -2.1×10^{-12} m/s, that is, zero to the tolerance of the solver.



Reading the figure

Element	Meaning
left panel, blue solid	the twelve-state vessel
left panel, orange dashed	a simple linear model of the same loop
right panel, orange dashed	that same linear model again
right panel, green solid	the same loop with the PI zero removed and nothing else changed

What the figure says

The integral term works. The speed settles exactly on 1.5 m/s, with a residual error of -2.1×10^{-12} m/s — the offset that section D could not remove is gone.

The surprise is the overshoot. The design asked for $\zeta = 0.7$, and the standard formula says a system with $\zeta = 0.7$ overshoots 4.60%. The vessel overshoots 8.15% — nearly double.

The left panel rules out the obvious suspect. If the vessel were more complicated than the design assumed, the blue and orange curves would separate. They do not: a simple linear model of the loop lands within 0.1 points of the twelve-state vessel. **The vessel is behaving exactly as the linear model says it should.** So the error is in the prediction, not in the plant.

The right panel shows where the prediction went wrong. Both curves there have **identical poles** — same ζ, same ω_n. The only difference is that the orange one contains the zero a PI controller unavoidably creates at $s = -K_i/K_p$, and the green one does not. That single difference moves the overshoot from 4.60% to 8.07%.

So ζ and ω_n describe only the **poles**, and a PI controller places a **zero** as well — one nobody asked for and the design procedure never mentions. The 4.60% formula was derived for a system with no zero at all, so it was never going to apply here.

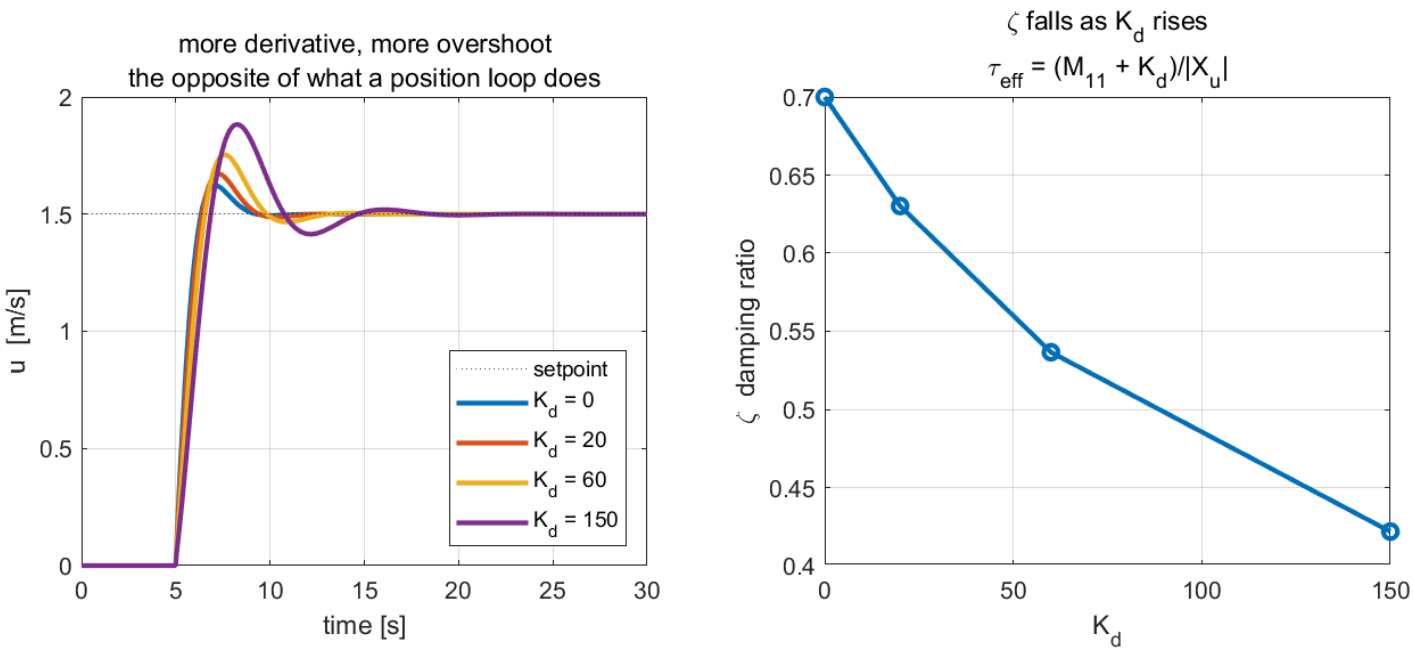
The damping ratio was designed correctly. The prediction made from it was not.

> ✎ **What to do about it**
>
> Pushing the zero further from the poles — a smaller K_i/K_p — makes the textbook figure accurate again, and costs a slower recovery from a load change. Week 8's reference model is the general remedy: shape the setpoint rather than argue with the closed-loop zeros.

Derivative

K_p and K_i held at the values above:

K_d	$M_{11} + K_d$ [kg]	ω_n	ζ	predicted M_p [%]	measured M_p [%]
0	85.50	1.5000	0.7000	8.07	8.15
20	105.50	1.3504	0.6302	11.60	11.48
60	145.50	1.1499	0.5366	17.39	16.88
150	235.50	0.9038	0.4218	26.56	25.50



Reading the figure

Element	Meaning
left panel	four step responses; overshoot grows monotonically with K_d
right panel	the damping ratio ζ computed from the effective mass $M_{11} + K_d$, falling as K_d rises

What the figure says

Derivative action makes this loop **worse**, and the figure leaves no room to argue about it. Overshoot rises with K_d — 8.15%, 11.48%, 16.88%, 25.50% — while ζ falls from 0.700 to 0.422, and the prediction tracks the measurement to within 1.1 points across the whole sweep.

The reason is where K_d lands when the control law is substituted into the surge equation:

$$(M_{11} + K_d)\dot{u} + |X_u|u = \dots$$

It sits **beside the mass**, not beside the damping. So $K_d = 150$ does not damp the vessel; it makes it **heavier**, from 85.5 to 235.5 kg — and a heavier vessel with unchanged damping is a less damped vessel.

Week 3 runs the identical term in an identical PID controller and gets the opposite result. The difference is not the controller and not the tuning. On a **velocity** loop the derivative of the controlled variable is an acceleration, and acceleration multiplies mass. On an **angle** loop it is a rate, and rate multiplies damping. **The term did not change; the axis did.**

The conclusion is not that derivative action is useless — it is that a term has to be substituted into the equation of motion before its effect can be assumed. Nothing here can be tuned away, so the practical answer on a speed loop is $K_d = 0$, which is what the rest of this week uses. Section I revisits the term on a plant that genuinely wants one.

F. Windup (25 min)

✎ To produce every figure in this section

script	`W02_F_windup.m`
model	`W02_surge_control.slx`
figure	`img/W02_result_windup.png`

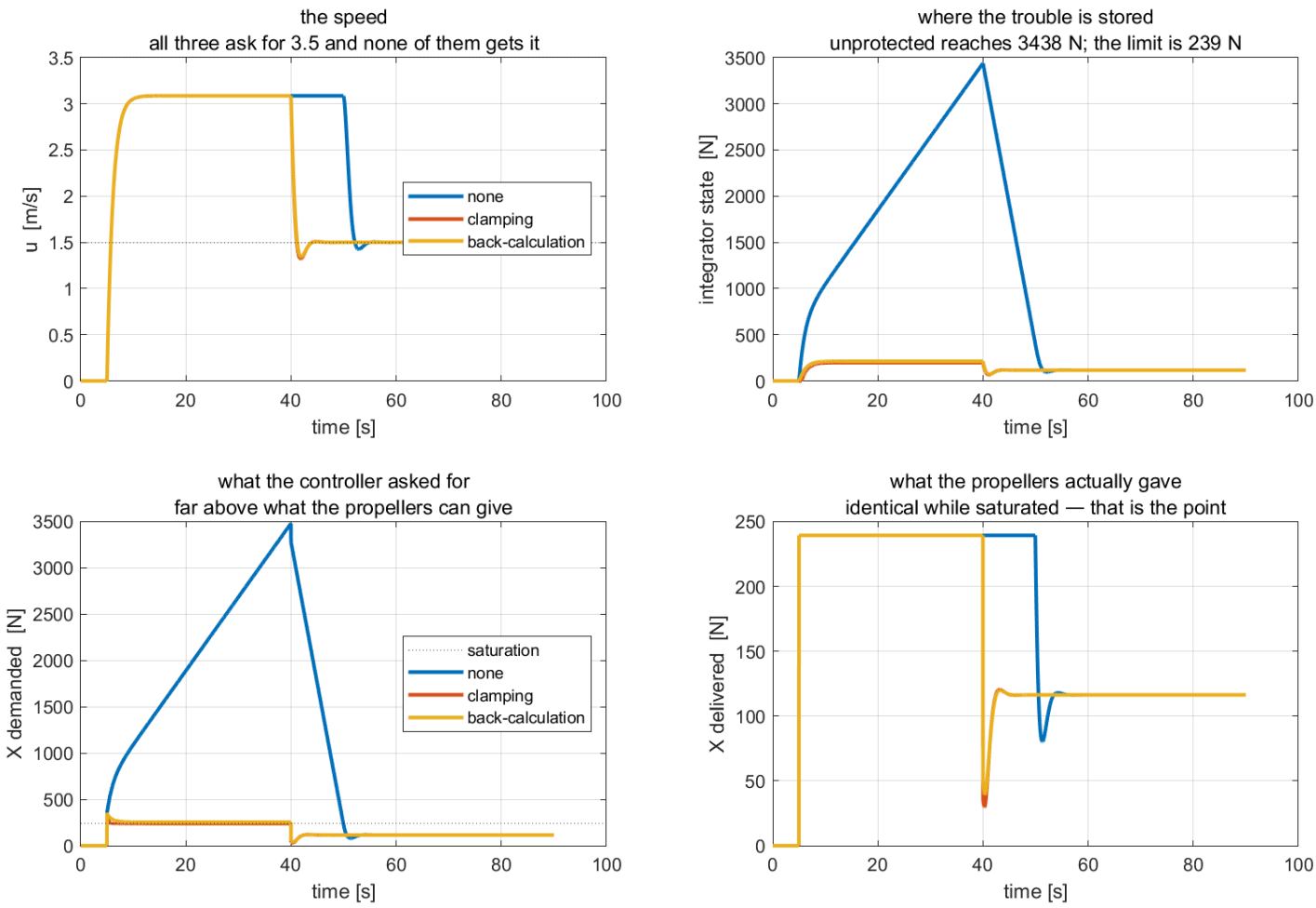
```
W02_0_setup                    % once per session
W02_F_windup
```

Opening the model and pressing **Run** produces the same figure: the scopes update as it runs and `StopFcn` draws the summary at the end.

- $u_d = 3.5$ m/s is applied at $t = 5$ s. It is **unreachable**: $u_{\max} = 3.0864$ m/s. At $t = 40$ s the demand drops to 1.5 m/s, which is reachable.

Anti-windup	peak I [N]	peak X_{cmd} [N]	recovery [s]
none	3438.2	3480.0	13.980
clamping	197.8	355.9	3.400
back-calculation	339.0	464.1	3.640

- **Recovery** is measured from $t = 40$ s to the last instant at which $|u - 1.5| > 0.03$ m/s, that is, a 2% band on the new setpoint. The averaging window is the whole interval $[40,\ 90]$ s.



Reading the figure

Element	Meaning
top left	the speed; without anti-windup the vessel holds 3.09 m/s for ten seconds after the demand has dropped
top right	the integrator state, reaching 3438 N against a largest useful value of 239 N
bottom left	the force the controller **asked for**, with the saturation limit dotted
bottom right	the force the propellers **delivered** — identical in all three runs while saturated

- The integrator accumulates 14.4 times the largest force the actuator can deliver. Every newton of that has to be integrated back out before the loop responds at all.
- Clamping and back-calculation differ little here — 3.40 s against 3.44 s. Clamping is slightly faster because it stops the integrator completely; back-calculation is smoother at the boundary and has one gain to choose. On this plant the choice is not important, which is itself worth knowing.

What the figure says

Four views of the same three runs. The two left panels are what happened and what was asked for; the two right panels are where the trouble was stored and what the water actually received.

For the first 40 s the three runs are **identical** in the top-left and bottom-right panels. Every controller sits on the limit, delivers 239 N, and drives the vessel to 3.09 m/s. Nothing whatsoever distinguishes them. They separate the instant the setpoint drops to a reachable 1.5 m/s: the protected runs recover in 3.4 s, the unprotected one takes 13.98 s and holds 3.09 m/s for ten seconds after being told to slow down.

The top-right panel is the whole diagnosis. While the demand is impossible the error never changes sign, so the integrator keeps accumulating — reaching 3438 N, a force the propellers can never produce. **The difference**

between the three runs lives entirely in the integrator and nowhere else.

Now compare the two bottom panels, because that pair carries the lesson. The bottom-left shows demands differing by a factor of fourteen. The bottom-right shows that the water felt **exactly the same force** in all three. A demand above saturation is not a control action; it is a number in a register. The loop was open, and no gain chosen from a closed-loop analysis can account for a loop that is not closed.

The two remedies both work here and work differently. Clamping stops the integrator wherever it happens to be; back-calculation steers it toward the value that would make the demand equal the limit. They settle at different integrator values, which is why raising the back-calculation gain never turns one into the other. Section H measures that on a plant simple enough to see it happen.

> **★ Nothing was wrong with the integrator**
>
> It did exactly what an integrator does. The loop had been opened by the actuator, and no gain chosen inside a closed-loop analysis can account for a loop that is not closed. Every one of the four experiments this week was predicted correctly by the linear model **except** where saturation intervened, and that is the boundary of linear design.

G. Hand-built against the library block (10 min)

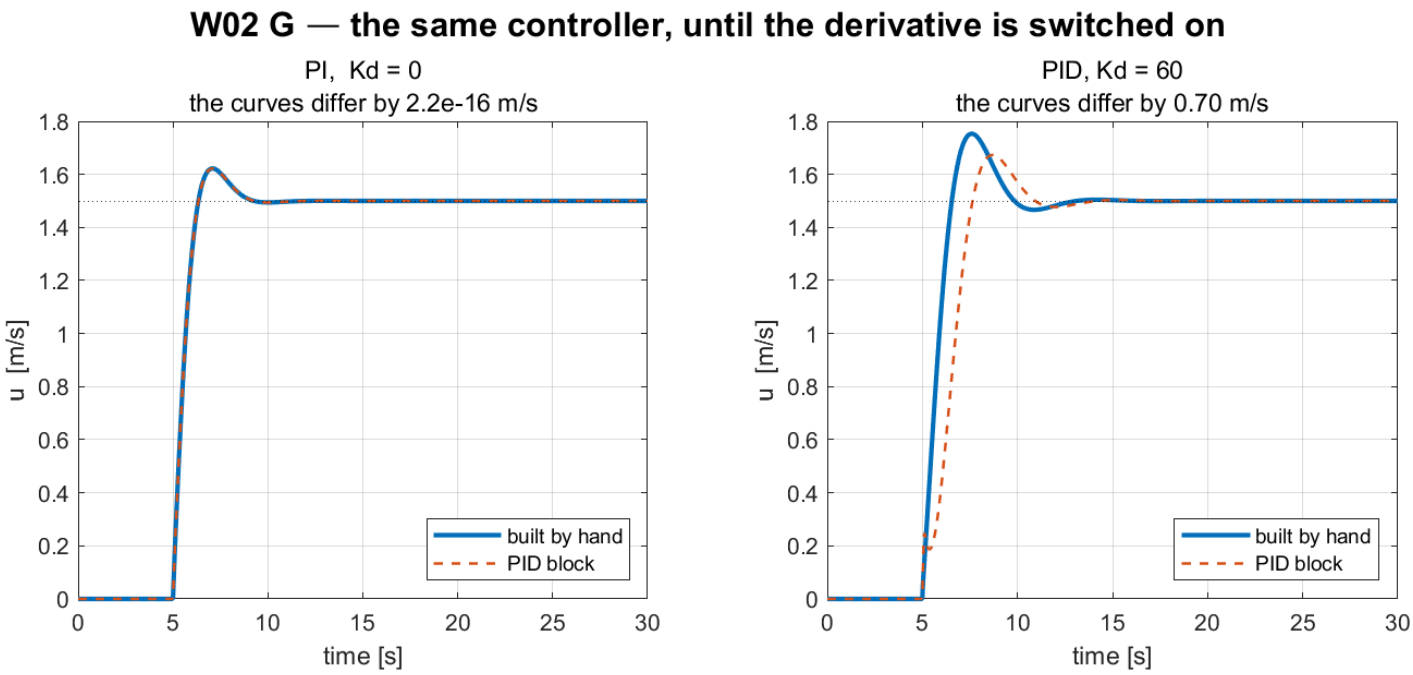
> **✎ To produce every figure in this section**
>
script	`W02_G_block_vs_handbuilt.m`
model	`W02_surge_control.slx`
figure	`img/W02_result_block.png`

```
W02_0_setup                    % once per session
W02_G_block_vs_handbuilt
```

Opening the model and pressing **Run** produces the same figure: the scopes update as it runs and `StopFcn` draws the summary at the end.

The same two runs, with `pid_mode = 0` and `pid_mode = 1`:

Case	largest gap in u [m/s]	M_p hand-built [%]	M_p library block [%]
PI, $K_d = 0$	2.2×10^{-16}	8.15	8.15
PID, $K_d = 60$	7.0×10^{-1}	16.88	11.54



Reading the figure

Element	Meaning
left panel	$K_d = 0$: the thick and dashed traces are one curve
right panel	$K_d = 60$: they separate, and the library block overshoots less

- With $K_d = 0$ the two agree to **machine precision**. Nine blocks and one library block compute the same thing, and the back-calculation written by hand in §2-6 is the back-calculation the block implements.
- With $K_d = 60$ they differ by 0.70 m/s at the peak. Neither is wrong. The library block differentiates the **error**, so a step in u_d passes through its derivative and produces a kick that the hand-built path — which differentiates the **measurement** — never sees.

What the figure says

The same plant, the same gains and the same setpoint, controlled twice — once by nine blocks assembled from the equations of §2-4, once by Simulink's PID Controller block. The two panels differ only in whether the derivative term is switched on.

In the left panel, with $K_d = 0$, the two traces are **one curve**. The largest difference anywhere in the run is 2.2×10^{-16} m/s, which is machine precision. Nine blocks and one block compute the same thing, and that agreement is what licenses using the library block for the rest of the course: it is not a different algorithm, only a shorter way of writing the same one.

In the right panel, with $K_d = 60$, they part company. The hand-built path peaks at 1.76 m/s and the block at 1.68 — overshoots of 16.88% against 11.54% — before converging again on the same 1.5 m/s.

The cause is one design choice, not a defect in either. The library block differentiates its **input**, which here is the error. The hand-built path differentiates the **measurement**. A step in u_d therefore passes straight through the block's derivative and produces a kick, while a measurement never steps, so the hand-built path sees nothing. **Both are correct implementations of a PID controller, which settles the more useful point: "a PID controller" is not a specific enough phrase to distinguish them.**

The gap appears only at a setpoint change and vanishes in steady state. A loop that mostly rejects disturbances will never notice it; a loop that mostly follows commands will notice a great deal. Simulink's two-degree-of-freedom PID block exists to weight the two paths separately, and is the general answer to the question this figure raises.

> ✎ **Which to use**
>
> The library block, in almost every case. It is one block, it is tested, and it offers discrete-time forms, external reset and tracking mode that would each take several more blocks by hand. Build it by hand once, to know what is inside it, and then stop.

H. The principle on its own (20 min)

> ✎ **To produce every figure in this section**

script	`W02_H_antiwindup_run.m`
model	`W02_H_antiwindup.slx`
figure	`img/W02_result_aw_principle.png`

```
W02_0_setup                    % once per session
W02_H_antiwindup_run
```

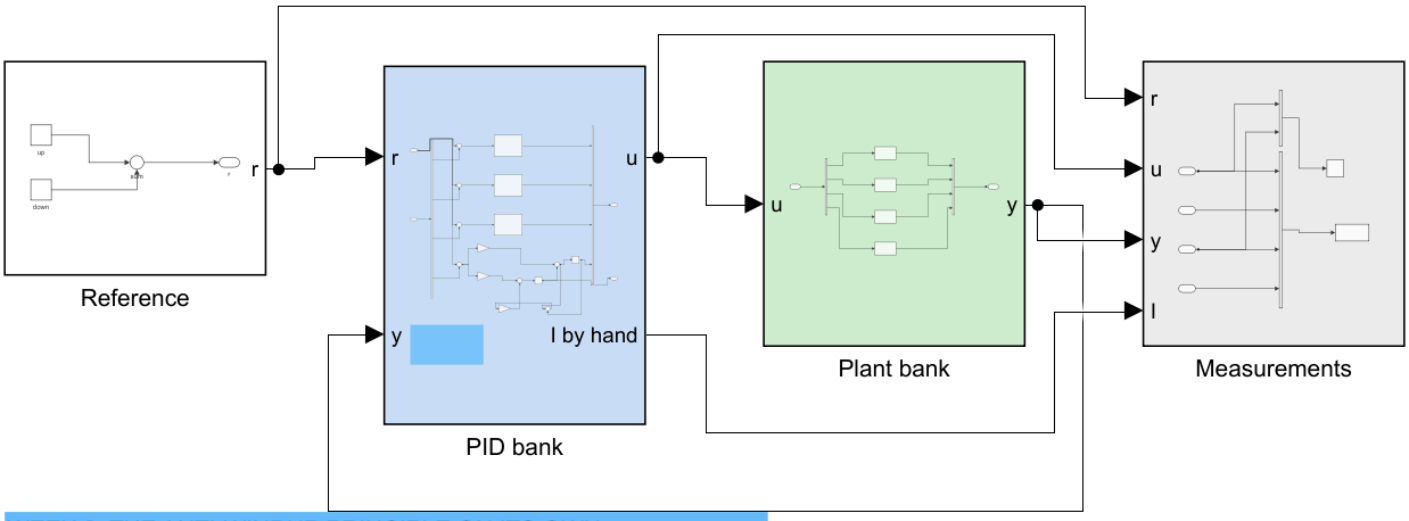
Opening the model and pressing **Run** produces the same figure: the scopes update as it runs and `StopFcn` draws the summary at the end.

- The vessel is a distraction from the mechanism. A second model strips everything away:

$$G(s) = \frac{1}{s+1}, \qquad u \in [-1,\ +1], \qquad \text{PI with } K_p = 1.8,\ K_i = 4.$$

- The DC gain is 1 and $|u| \le 1$, so the largest reachable output is $y = 1$. The reference is stepped to 2 — unreachable — held for 14 s, then dropped to 0.5, which is reachable.

```
W02_H_build_antiwindup
W02_H_antiwindup_run
```



WEEK 2, THE ANTI-WINDUP PRINCIPLE ON ITS OWN

No vessel, no propellers, no twelve states. A first-order plant, a limit, and an integrator that does not know about the limit:

G(s) = 1 / (s + 1), u in [-1, +1], PI control.

The DC gain is 1 and |u| <= 1, so the largest reachable output is y = 1. The reference is stepped to 2, which CANNOT be reached, held, and then dropped to 0.5, which can.

FOUR CONTROLLERS, IDENTICAL GAINS, ONE PLANT EACH

1 Simulink PID block, anti-windup = none
2 Simulink PID block, anti-windup = clamping
3 Simulink PID block, anti-windup = back-calculation, gain Kb
4 the same back-calculation built by hand

Rows 3 and 4 must agree to machine precision. That agreement is what licenses the claim that the library block does what the theory says.

WHAT TO WATCH

Not the rise to y = 1 - all four are identical there. Watch what happens AFTER t = 15 s, when the reference becomes reachable again. Row 1 sits at y = 1 for several seconds doing nothing, because the integrator has to unwind a charge it should never have accumulated.

Then open PID bank and read the annotation on row 4.

Reading the figure

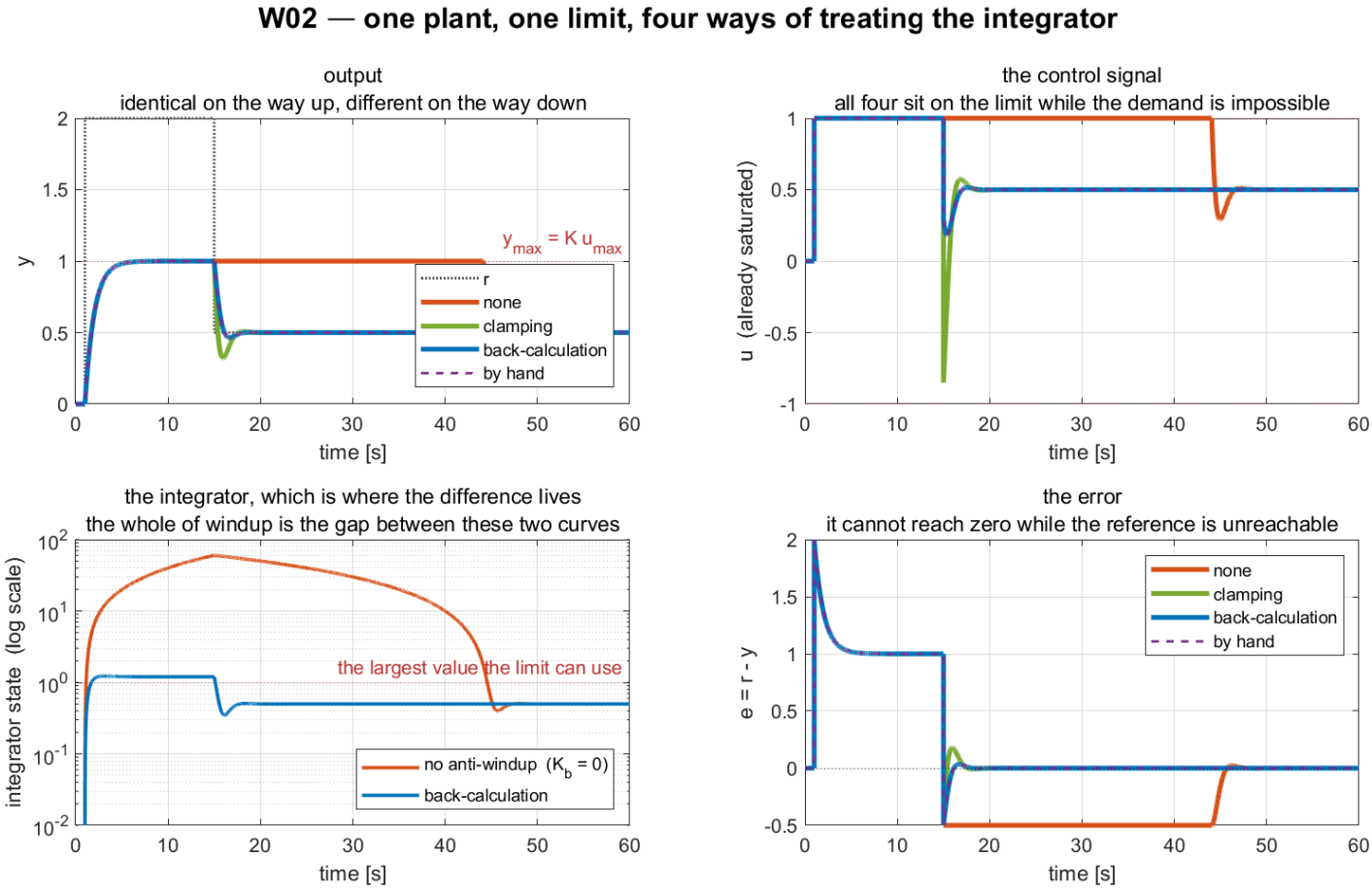
Element	Meaning
`Reference` (white)	the step up to an impossible value, then down to a possible one
`PID bank` (blue)	four controllers with identical gains: three library blocks with the three anti-windup settings, and the fourth built by hand
`Plant bank` (green)	four identical copies of $1/(s+1)$, one per controller
`Measurements` (grey)	the log

- The fourth row exists so that the **integrator state is a signal**. The library block does not expose it, and the integrator is where the whole phenomenon lives.

Results

Anti-windup	time on the limit [s]	recovery [s]	y at $t = 20$ s
none	43.05	32.04	1.0000
clamping	14.00	2.380	0.4997
back-calculation	14.00	2.645	0.5003
the same, by hand	14.00	2.645	0.5003

- **Recovery** is the time after $t = 15$ s for y to enter and stay within a 2% band on 0.5.



Reading the figure

Element	Meaning
top left	the output; all four are one curve on the way up
top right	the control signal; all four sit on the limit for the same 14 s
bottom left	the integrator state on a log scale, with and without anti-windup
bottom right	the error, which cannot reach zero while the reference is unreachable

- **All four are identical on the way up.** While the demand is impossible every scheme sits on the limit and produces $y = 1$. Nothing distinguishes them until $t = 15$ s.
- The bottom-left panel is the whole phenomenon. Without anti-windup the integrator winds to 60.00, against a largest useful value of 1.0 and against 1.225 with back-calculation — a factor of 49. It then takes 30 s to unwind, and the loop does nothing for all of it.
- Rows 3 and 4 agree to **exactly zero**. The library block and the hand-built path are the same algorithm, so the theory of §2-6 is a correct description of what the block does.

What the figure says

For the first 14 s all four curves coincide in three of the four panels. The output holds at $y = 1$, the control sits flat on the limit, the error sits flat at $+1$. Only the integrator panel separates them — and it separates them by a factor of 49: 60.00 unprotected against 1.225 with back-calculation.

An integrator integrates. While the reference is unreachable the error cannot change sign, so the integral grows without bound, into a quantity the actuator has no way to use. **The trouble is not that the integrator misbehaved; it is that the loop was open and the integrator was not told.**

That panel is on a log scale for a reason. Drawn linearly, the protected curve would lie flat against the axis and the reader would see one curve and a wall. On a log scale both are visible, and what it shows is not merely a smaller number but an integrator that **stays inside the range the actuator can act on** for the whole run.

The setpoint here is deliberately impossible. A scheme that is invisible for 14 s and decisive at second 15 cannot be judged on a reachable command — and when second 15 arrives, the unprotected loop needs 32.04 s to settle while the others need about 2.5.

> **★ Choosing K_{aw}**
>
> The usual starting point is $K_{aw} = 1/\tau$, which on this plant is 1 and gives a 3.49 s recovery. Sweeping it finds a shallow minimum near $K_{aw} = 5$ at 2.375 s: below that the integrator is not emptied fast enough, above it the loop leaves the limit so abruptly that undershoot grows from 4.60% to 43.96%. The rule of thumb is a reasonable default and it is not the optimum.
>
> Raising K_{aw} does **not** turn back-calculation into clamping. Clamping stops the integrator wherever it happens to be; back-calculation steers it to the value that makes the demand equal the limit. Those are different fixed points, not two ends of one scale — which is why the swept curve dips *below* clamping's 2.380 s and comes back up rather than approaching it as an asymptote.

I. The pseudo-derivative, on a plant that wants one (25 min)

> **✎ To produce every figure in this section**
>
script	`W02_I_pseudo_derivative_run.m`
model	`W02_I_pseudo_derivative.slx`
figure	`img/W02_result_pd.png`

```
W02_0_setup                    % once per session
W02_I_pseudo_derivative_run
```

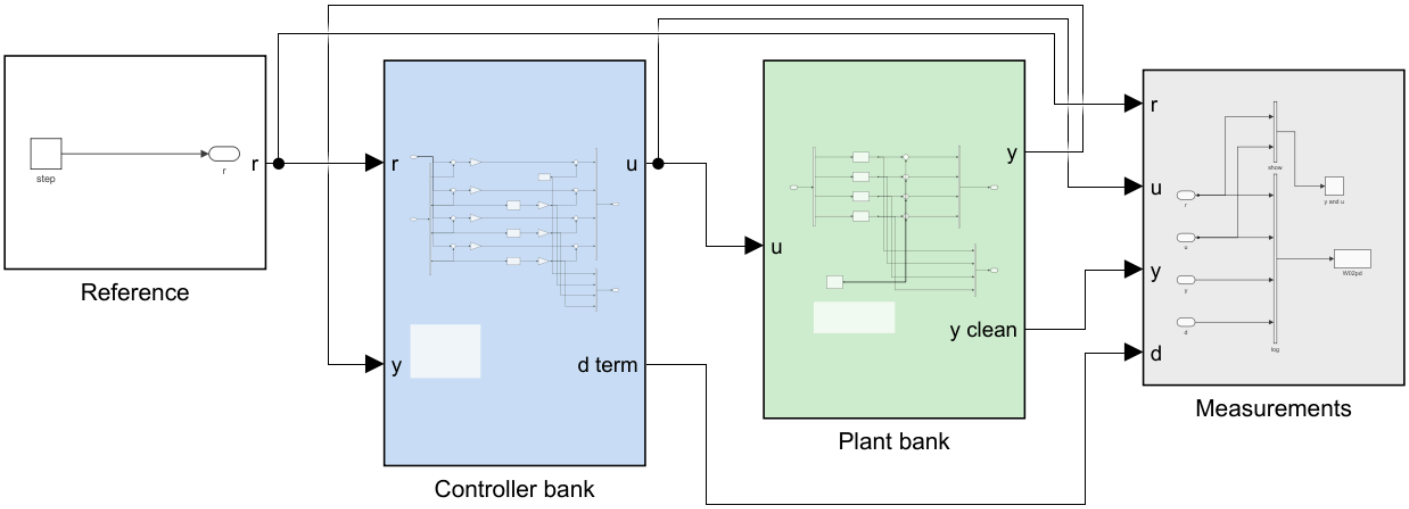
Opening the model and pressing **Run** produces the same figure: the scopes update as it runs and `StopFcn` draws the summary at the end.

- §2-4 found that on the surge axis the derivative term acts as a **mass** and makes things worse, so nothing in sections A to G shows what the derivative filter is *for*.
- A third model uses a plant where derivative action is genuinely wanted:

$$G(s) = \frac{1}{s^2 + 0.4\,s}, \qquad K_p = 4,\ K_d = 2, \qquad \text{measurement noise } \sigma = 0.01.$$

- Under proportional control alone the closed loop is $s^2 + 0.4s + K_p$, so $\zeta = 0.2/\sqrt{K_p} = 0.1$. It rings badly. Derivative action is the cure, which makes *how to take the derivative* a real question.

```
W02_I_build_pseudo_derivative
W02_I_pseudo_derivative_run
```



WEEK 2, SECTION I - THE PSEUDO-DERIVATIVE

A PID controller never contains a differentiator. This model shows why, on a plant where derivative action is actually wanted.

G(s) = 1 / (s^2 + 0.4 s) zeta = 0.1 under P control alone

FOUR ROWS, SAME GAINS, SAME NOISE

1 P only rings, but the actuator is quiet
2 Kd s ideal damps well, actuator unusable
3 Kd N1 s/(s+N1) pseudo N1 large - still noisy
4 Kd N2 s/(s+N2) pseudo N2 small - quiet, slightly slower

THE MECHANISM

Differentiation has gain |w|: it multiplies every component of the signal by its own frequency. Noise is the fastest thing in the measurement, so differentiation finds it and amplifies it more than anything else. The pseudo-derivative is the same operator with its gain capped at N, so N is the knob that decides how much of the noise reaches the actuator.

The cost is phase lag below N. Choosing N is therefore a trade, and the runner measures both sides of it.

Edit pd_Kp, pd_Kd, pd_N1, pd_N2 and pd_noise in W02_0_setup.m.

Reading the figure

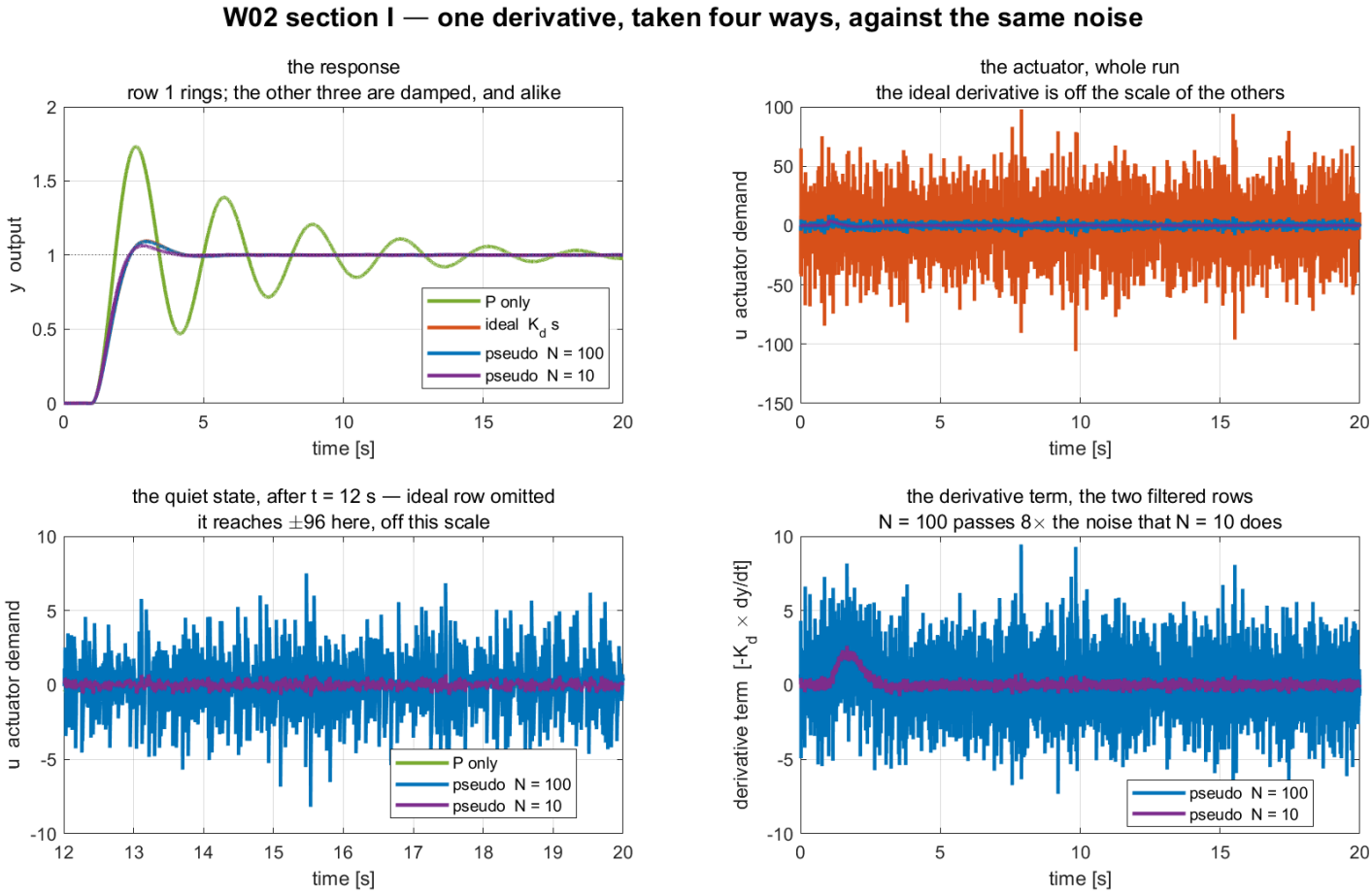
Element	Meaning
`Reference` (white)	one step, at $t = 1$ s
`Controller bank` (blue)	four controllers, identical except for the block in the derivative slot: none, ideal s, and the pseudo-derivative at two values of N
`Plant bank` (green)	four copies of $G(s)$ and **one** noise source shared by all four
`Measurements` (grey)	the log, which records the clean output so the plots are not themselves noisy

> **★ One noise source, not four**
>
> Four independent noise generators would make the four control signals differ for two reasons at once, and the comparison would prove nothing. Sharing one realisation is what licenses the claim that the differences below are caused by the derivative implementation alone.

Results

Derivative	overshoot	settling	RMS(u) quiet	max $\lvert u \rvert$
P only	73.22%	19.00 s	0.1814	4.04
ideal, $K_d s$	9.54%	2.99 s	8.7452	105.94
pseudo, $N = 100$	9.10%	2.97 s	1.6780	9.61
pseudo, $N = 10$	6.28%	2.63 s	0.2348	4.26

- **RMS(u) quiet** is the standard deviation of the actuator demand after $t = 12$ s, when the setpoint has been reached and everything remaining in the signal is noise.



Reading the figure

Element	Meaning
top left	the outputs. Row 1 rings for the whole run; the other three are damped and nearly identical
top right	the actuator demand. The ideal derivative reaches ± 106 on a plant whose steady demand is 4
bottom left	the quiet state, **with the ideal row omitted** so the other three are visible at all
bottom right	the derivative term alone, for the two filtered rows

What the figure says

Look at the two top panels together, because the contrast between them is the whole section.

In the top-left panel the three derivative rows are almost indistinguishable. Whatever the difference between them is, it is not the quality of the control — all three damp the plant equally well.

In the top-right panel they are nothing alike. The ideal derivative swings ± 106 against a steady demand of 4 — **twenty-six times the useful value**, and every bit of it noise.

Differentiation has gain $|j\omega|$, which grows without bound, so it finds the fastest thing in the measurement and multiplies it by the largest number available. The fastest thing in any measurement is the noise. The pseudo-derivative $Ns/(s+N)$ has the same gain below N and levels off above it: **the same operator, with a ceiling on how much it may amplify.**

The bottom-left panel leaves the ideal row out on purpose — included, it flattens the other three onto the axis and nothing can be read. Its peak of ± 96 is printed on the panel instead, so nothing is hidden by the omission.

One control experiment settles what the difference actually is. Repeating all four runs with the noise generator switched off collapses every quiet-state RMS to below 2×10^{-6} while the overshoots move by less than 0.1 point. So the factor of 37 between the ideal and the filtered rows **was noise and nothing else** — not the filter's phase lag, which would have shown up here and did not.

> **★ Where to put N**
>
> Sweeping N from 2 to 500 shows two different mechanisms, not one trade-off. Below $N \approx 5$ the filter is still shaping the response and overshoot improves with N, from 14.72% to 4.77%. Above it the response has flattened — from $N = 100$ to $N = 500$ overshoot moves by 0.3 points — while actuator noise keeps growing, from 1.68 to 5.50.
>
> Large N is therefore not a cautious default; above the bandwidth it buys **nothing** and charges for it. The closed-loop bandwidth here is $\omega_n = \sqrt{K_p} = 2.0$ rad/s and the measured best N is 5. **Place N just above the closed-loop bandwidth and stop.**

Why this matters back on the vessel

- Week 3 controls heading, where the derivative term supplies genuine damping and is therefore indispensable. Its input is a yaw rate from an IMU, which is a noisy measurement.
- Everything in this section applies there unchanged, with one simplification: MSS convention feeds back the measured rate r directly instead of differentiating ψ. **The cleanest derivative is the one that was never taken** — if the state is already measured, no filter is needed and no noise is amplified.

Summary

Week Summary

Step	What was done	How it was verified
1	Identified the surge plant from a step response	K_u exact to four decimals; $\tau = 1.1200$ s against 1.1025 s predicted
2	Derived and measured the type 0 steady-state error	43.68, 13.43, 3.73 per cent at $K_p = 100$, 500, 2000, matching the final value theorem
3	Placed the closed-loop poles at $\zeta = 0.7$, $\omega_n = 1.5$	overshoot 8.15% measured against 8.07% predicted **with** the PI zero, 4.60% without it
4	Substituted the derivative term into the equation of motion	$\tau_{\text{eff}} = (M_{11} + K_d)/\lvert X_u \rvert$; overshoot predicted within 1.1 points over $K_d \in [0, 150]$
5	Established the exact speed ceiling	$u_{\max} = U_{\max} = 3.0864$ m/s, from $X_{\max} = 24.4g$ and $X_u = -24.4g/U_{\max}$
6	Measured windup and two remedies	integrator peak 3438 N; recovery 13.98 s against 3.40 s and 3.64 s
7	Compared the hand-built controller with the library PID block	identical to 2×10^{-16} m/s at $K_d = 0$; 0.18 m/s apart at $K_d = 60$
8	Isolated the principle on $1/(s+1)$	integrator peak 60.00 against 1.225; recovery 32.04 s against 2.645 s
9	Swept the back-calculation gain	fastest recovery at $K_{\text{aw}} = 5$; undershoot $4.60 \to 43.96$ per cent across the sweep
10	Measured what the derivative filter buys	ideal derivative: actuator RMS 8.75 and peak 105.94; at $N = 10$, RMS 0.2348 with *less* overshoot
11	Swept the filter coefficient N	best overshoot at $N = 5$; above it overshoot flat within 0.3 points while RMS(u) grew $3\times$

Progress Check

Theory

- [ ] Able to state why the surge loop is type 0 without computing anything
- [ ] Able to derive $e_{ss}/u_d = 1/(1 + K_p K_u)$ from the final value theorem
- [ ] Able to invert ζ and ω_n for K_p and K_i, and to locate the PI zero
- [ ] Able to explain why K_d raises the overshoot on this axis and will not on the next

Laboratory

- [ ] `W02_0_setup` printed $\zeta = 0.7000$ and $\omega_n = 1.5000$
- [ ] each of `W02_C_…` through `W02_G_…` ran on its own and wrote its figure into `W02_simulink/img/`
- [ ] `W02_H_antiwindup_run` and `W02_I_pseudo_derivative_run` completed and wrote four more
- [ ] The open-loop identification was run and τ read from the 63.2% crossing

Recorded observations

- [ ] The measured steady-state error for three gains, against the predicted curve
- [ ] The overshoot with and without the PI zero in the prediction

- [ ] The integrator peak and the recovery time for all three anti-windup settings, with the tolerance band stated
- [ ] The gap between the hand-built controller and the library block, at $K_d = 0$ and $K_d = 60$
- [ ] The K_{extaw} at which recovery is fastest, and the undershoot it costs

---

In-class laboratory — close the speed loop by hand

The second hour of the Week 2 session is spent building, in Simulink, the loop this lecture built from a script. Three problems, one hour, with a checker that compares the result against the numbers measured above.

```
cd lectures/W02_simulink/problems
W02_P1_start                % creates W02_P1.slx — hull and thrust map only
W02_check(1)                % run this whenever, as often as needed
```

	Problem	Time	The number it must reproduce
1	The open loop — a constant force in, a log out	20 min	$u_{ss} = K_u X$ exactly: 0.6447, 1.2894, 2.5788 m/s
2	Proportional control — and the error that never closes	20 min	0.8448 m/s at $K_p = 100$, against $u_d = 1.5$
3	Add the integrator — and find what it cost	20 min	error $\to 0$, and overshoot **appears**

- The full problem sheet is `W02_simulink/problems/README.md`, and reference answers are in `W02_simulink/solutions/`.
- The problem sheet carries **the result graphs a correct model produces**, so a plot can be compared against a plot and not only against a number.
- The hull and the thrust map are provided. Everything between the reference and the plant is built by hand.

Assignment 2

- **Due**: before the Week 3 session
- **Submit**: the modified `W02_0_setup.m`, a derivation, the numbers requested below, and two figures

① Requirements

1. Derive e_{ss}/u_d for the proportional loop from the final value theorem, showing the limit explicitly. Then determine, by hand, the gain $K_p^\star$ that leaves exactly 5% steady-state error at $u_d = 1.5$ m/s.
2. Determine the demanded steady force X_{ss} at $K_p^\star$ and state whether it lies inside the attainable control set of Appendix A1.
3. Design a PI controller for $\zeta = 1.0$ (critically damped poles) at $\omega_n = 1.2$ rad/s. State K_p, K_i, the pole locations and the zero location, and **predict the overshoot including the zero**. A critically damped pole pair with a zero does not give zero overshoot; say what it does give and why.
4. Determine, by hand, the largest step in u_d from rest that the design of ③ can command **without** the actuator saturating at any instant.

② Verification — mandatory

> **★ A claim is not a result. Numbers are required, with the tolerance band and the averaging window stated**

1. Run the proportional loop at $K_p^\star$ and report the measured steady-state error to four decimal places, with the averaging window stated.

2. Run the PI design of ①.3 and report the measured overshoot and 2% settling time. Compare with both predictions — poles only, and poles with the zero — and state which one the plant follows.
3. Apply the step of ①.4 and report the peak X_{cmd} and peak X_{sat}. State whether saturation occurred. Then apply a step 20% larger and repeat.
4. With the design of ①.3 and $u_d = 3.5$ m/s held for 35 s, sweep $K_{\text{aw}} \in \{0.2, 0.5, 1/T_u, 2, 5\}$ and report the recovery time for each. Produce **one figure** of recovery time against K_{aw}.
5. Produce **one figure** comparing the response of ①.3 with $K_d = 0$ and with $K_d = 40$, and report both overshoots.

③ Analysis (8–12 lines)

- Item ②.4 has a minimum somewhere in the sweep, or it does not. State which, and explain the behaviour at both ends: what happens as $K_{\text{aw}} \to 0$, and what happens when K_{aw} is made large. Relate the large-K_{aw} behaviour to the clamping scheme, which is the limiting case. Then state, in one sentence, what would remove the need for anti-windup entirely in this experiment.

Grading

Criterion	Weight
Derivations in ① are complete, including the overshoot prediction with the zero	20%
Verification performed and numbers reported with bands and windows stated	40%
Both figures are correct and readable	15%
Analysis in ③ explains both ends of the sweep and identifies the real remedy	25%

Troubleshooting

Symptom	Cause	Fix
The speed settles far below u_d with $K_i > 0$	`aw_mode = 2` with `Ki = 0` charges the integrator during saturation and can never discharge it	the model guards against this; if a modified version does not, set `aw_mode = 0` whenever `Ki = 0`
The integrator output is identically zero	an input of the `anti-windup` block is unconnected, so `Ki` arrives as 0	rebuild with `W02_1_build_surge_control`
The measured overshoot is far above the tabulated value for the design ζ	expected	a PI controller adds a zero at $-K_i/K_p$; compare against the linear model, not against the table
Adding K_d makes the response worse	expected on this axis	K_d adds to M_{11}; see §2-4
The vessel never reaches u_d however large the gain	u_d exceeds $u_{\max} = 3.0864$ m/s	no controller can pass this limit; reduce the setpoint
The response is unchanged when `Kp` is edited in the block dialog	the block reads the workspace variable	edit `W02_0_setup.m` and run it
`stepinfo` is undefined	Control System Toolbox is not licensed	the simulation results are unaffected; only the printed comparison in section E requires it
The library block and the hand-built path disagree at $K_d > 0$	expected	the block differentiates the error, the hand-built path the measurement; see §2-7
The library block's anti-windup setting cannot be changed from a variable	it is a dialog choice, not a signal	the hand-built path selects with `aw_mode`; the demonstration model of §H has one block per setting
`W02_H_antiwindup_run` reports a recovery equal to the run length	the run ended before the loop recovered	lengthen `aw_T`; the default 60 s is enough for $K_{\text{aw}} \geq 0.2$

References

Primary

- Fossen, T. I. *Handbook of Marine Craft Hydrodynamics and Motion Control*, 2nd ed. §12.2 (PID control of marine craft) and §12.2.6 (integrator anti-windup).
- Åström, K. J. and Hägglund, T. *Advanced PID Control*. ISA, 2006. Chapter 3, for the derivative filter and the two anti-windup schemes.
- MSS toolbox, `Tools/MSS/VESSELS/otter.m` — the damping and saturation constants, lines 65 and 91–98.

Course files

- `W02_simulink/W02_1_build_surge_control.m` — the model generator
- `W02_simulink/W02_0_setup.m` — the parameters, including the PI design equations
- `W02_simulink/W02_C_identify_plant.m` … `W02_G_block_vs_handbuilt.m` — one script per lecture section
- `W02_simulink/W02_plot.m` — the summary figure, called by both the runner and the model's `StopFcn`
- `W02_simulink/W02_animate.m` — the live view

- `W02_simulink/W02_H_build_antiwindup.m` — the standalone demonstration model of §H
- `W02_simulink/W02_H_antiwindup_run.m`, `W02_aw_plot.m` — section H, its experiments and figures

Next Week

- **Week 3 — Heading Control**
- The heading axis contains a free integrator, $\psi = \int r$, so the loop is **type 1** and proportional action alone leaves no steady-state error. Everything §2-2 established is reversed by one structural change.
- The derivative term reappears, this time acting on the yaw rate, and this time it damps.
- The angle ψ wraps, and the smallest-signed-angle function `ssa` is what keeps a controller from steering the long way round.
- Preparation: bring $M_{66} = 42.65$ kg·m² and $N_r = -42.65$ from Week 1, and the finding of §2-4 about where a derivative term goes.